Campus Manager

A Smart University Management System

A PROJECT REPORT

Submitted by

Sanjeevani Sharma(24MCA20213)
Siddhant Kumar (24MCA20241)
Raja Kumar(24MCA20229)
Prabhakar Singh(24MCA2020071)

Submitted in partial fulfilment for the award of the degree of

Master of Computer Applications

CHANDIGARH UNIVERSITY

BONAFIDE CERTIFICATE

Certified that this project report "Campus Manager" is the Bonafide work Of "Raja Kumar, Siddhant Kumar, Sanjeevani Sharma and Prabhakar Singh" who carried out the project work under my/our supervision.

SIGNATURE

Dr. Amanpreet Kaur Sandhu
HEAD OF THE DEPARTMENT
(University Institute of Computing)

SIGNATURE

Mr. Rajan Saluja
SUPERVISOR
(Assistant professor)
(University Institute of Computing)

Submitted for the project viva-voce examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION.....4-6
1.1 Need Identification
1.2 Identification of Problem
1.3 Identification of Tasks
1.4 Timeline of the Project
1.5 Organization of the Report

CHAPTER 2: BACKGROUND STUDY..... 7-16
2.1 Timeline of the Reported Problem
2.2 Proposed Solution
2.3 Bibliometric Analysis
2.4 Review Summary
2.5 Problem Definition
2.6 Goals / Objectives
2.7 Justification of Technology Selection

CHAPTER 3: SYSTEM ANALYSIS 16-28
3.1 Existing System Analysis
3.2 Limitations of Existing System
3.3 Proposed System Overview
3.4 Feasibility Study
3.4.1 Technical Feasibility
3.4.2 Economic Feasibility
3.4.3 Operational Feasibility
3.5 Requirement Analysis
3.5.1 Functional Requirements
3.5.2 Non-Functional Requirements
3.6 System Users and Roles

CHAPTER 4: SYSTEM DESIGN..... 28-46
4.1 System Architecture
4.2 Data Flow Diagram (DFD)
4.2.1 Level 0 DFD
4.2.2 Level 1 DFD
4.3 Entity Relationship Diagram (ER Diagram)
4.4 Database Design
4.5 UML Diagrams
4.5.1 Use Case Diagram
4.5.2 Sequence Diagram
4.6 User Interface Design
4.7 Design Constraints

CHAPTER 5: IMPLEMENTATION AND CODING 46–70
5.1 Development Environment
5.2 Frontend Implementation
5.3 Backend Implementation
5.4 Database Implementation
5.5 Module Description
5.5.1 Authentication Module
5.5.2 Department Module
5.5.3 Course Module

5.5.4 Module Management
5.6 Code Snippets
5.6.1 Login API Code
5.6.2 Course API Code
5.6.3 Database Models

CHAPTER 6: RESULT ANALYSIS AND VALIDATION.....70–88
6.1 System Testing
6.2 Output Screens
6.3 Performance Evaluation
6.4 Validation of Results

CHAPTER 7: CONCLUSION AND FUTURE WORK.....88–91
7.1 Conclusion
7.2 Future Enhancements

CHAPTER 1: INTRODUCTION

1.1 Need Identification:

In the modern digital age, educational institutions are growing in terms of the number of students, academic programs, and administrative activities. Managing the large amount of data is becoming increasingly difficult and inefficient using traditional methods, as they are largely based on paperwork and separate software applications. The traditional method is often associated with data duplication, human mistakes, and delays in data processing, as well as ineffective communication among the stakeholders.

In educational institutions, the need is to develop an efficient system that can effectively handle data related to the students, faculty, and administrative activities in an integrated manner. The absence of an efficient system often leads to the disintegration of data storage, making it difficult to access the data when required.

The requirement for a University Management System stems from the growing need for a system to automate, accuracy, and accessibility of data in a timely manner. A smart system like Campus Manager has the potential to reduce manual intervention, improve overall efficiency, and facilitate communication among students, faculty, and staff. It also helps in proper organization, security, and scalability, which are critical requirements for a modern educational institution.

Hence, the development of Campus Manager System is important to transform traditional campus operations into a digital, efficient, and user-friendly platform.

1.2 Identification of Problem

Yet, a number of educational institutions still rely on manual or obsolete methods to run their affairs, even though technology is continually improving. This, therefore, brings about a number of serious problems, impacting educational and administrative performance significantly.

First and foremost, one of the major problems is that manual methods are time-consuming. For instance, keeping track of students' records, attendance, and courses is a time-consuming task. Moreover, there is a high possibility that duplicate and inconsistent information may exist, especially since different departments may have separate records that are not synchronized.

Another significant problem is that educational institutions lack a unified system to manage information. The information is often stored in different systems or even physically, and this brings about difficulties in retrieving and managing the information**.**

Communication among all stakeholders is also a problem. Students, teachers, and management often encounter problems when trying to access updates, notices, and academic information in a timely manner.

In addition, the current systems do not have adequate security features, and this might result in unauthorized access to important information. This has therefore emphasized the importance of a reliable, secure, and integrated system.

The Campus Manager project has been initiated to overcome all these problems and create a central, automated, and secure system for efficiently handling all activities related to the campus.

1.3 Identification of Tasks

In order to solve the problems that have been identified and to meet the objectives of the project, a number of tasks are required to be performed. The tasks are aimed at ensuring that the development process is a success and that the system is implemented appropriately.

The first task is to analyze the requirements, where the requirements of the users, such as the administrators, the faculty, and the students, are analyzed. This is important since it ensures that the requirements are well understood and that the functionalities are known for the system to be developed.

The next task is to design the system, where the application is designed, including the database and the interface. This ensures that the system is efficient and that the application is user-friendly.

The next task after the design is the development, where the application is developed, including the frontend and the backend. The frontend is aimed at ensuring that the application is interactive, while the backend is aimed at ensuring that the business logic is implemented appropriately.

The next task is to integrate the system, where all the components, such as the authentication, the departments, the courses, and the users, are integrated into a single system.

Next, testing and debugging are carried out to correct errors in the system. This helps to ensure that the system operates effectively and serves its purpose adequately.

Lastly, the system is implemented, and documentation is developed. This includes developing user and technical manuals, among others, to aid in maintenance and upgrading of the system in future.

1.4 Timeline of the Project:

The development of the Campus Manager project is done with a structured time frame to ensure the achievement of systematic progress. The project is divided into several phases, each allocated a specific period of time.

Firstly, the project commences with requirement analysis and planning. This is followed by the system design phase, which includes database modeling and interface design.

The development phase is the next phase in the project. During this phase, the functionalities of the system are developed. This includes the creation of modules for user authentication, department management, course management, etc.

Finally, the integration and testing phase is done. During this phase, the integration of the entire project is done. Any errors that occur during the testing phase are corrected in the debugging phase.

The last phase includes documentation and deployment, whereby the system is ready for presentation and submission. Proper documentation helps in ensuring that the system can be easily understood and maintained.

The above timeline helps in effectively managing tasks, and the project can be completed within a specified period.

1.5 Organization of the Report:

The project report is divided into various chapters, each dealing with a particular feature of the Campus Manager system.

Chapter 1 is dedicated to the introduction to the project, where the need for the project, problem identification, tasks involved, and time required for the project are discussed.

Chapter 2 is dedicated to the background study, where literature review, problem definition, objectives, and analysis of the existing system are discussed.

Chapter 3 is dedicated to system analysis, where the analysis of the existing system is done, and the new system is defined, including its feasibility and requirements.

Chapter 4 is dedicated to system design, where the design of the system, including its architecture, DFD, database design, and user interface, is discussed.

Chapter 5 is dedicated to the implementation and coding part of the project, where the tools, modules, and structure of the code are discussed.

Chapter 6 is dedicated to the results, system output, and validation of the system developed.

Chapter 7 is dedicated to the conclusion, where the project is concluded, and future enhancements and possible improvements are discussed.

The project is well-structured and ensures a clear understanding of the project and its development and implementation of the Campus Manager system.

CHAPTER 2: BACKGROUND STUDY

2.1 Timeline of the Reported Problem:

The educational institutions' management process has undergone a significant change in recent times. Earlier, all the academic and administrative activities of the students, such as maintaining student records and attendance records and examination records, were handled manually with the help of paper-based systems. Even though these systems were suitable for handling a few students and academic activities, they are no longer efficient with the increasing number of students and academic activities.

With the advent of technology and the use of computer systems in handling the educational institutions' activities, the systems were mainly computer-based and handled various activities like student attendance and student records. However, these systems are not integrated with one another, and the data is scattered and inconsistent.

Recently, the use of web-based systems has been implemented to overcome the disadvantages of the computer-based systems and the standalone systems. However, the educational institutions are still facing problems like poor system integration and poor scalability and user-friendliness of the systems.

Therefore, the need of the hour is a system that can integrate the entire campus with a single system and can efficiently handle the entire campus activities with the help of a unified and intelligent system like the Campus Manager system.

2.2 Proposed Solution:

In order to address the challenges identified in the existing system, the proposed solution is the development of a Campus Manager – Smart Campus Management System.

Description of the Solution:

The proposed system will be a web-based application, which will provide an integrated platform for the academic and administrative requirements of the campus.

Features of the Solution:

The system will provide a centralized database in which all the information pertaining to the students, faculty members, courses, departments, etc., will be stored.

The system will provide the facility of user authentication, which will ensure that the users are able to access only the information they need.

The features of the Campus Manager – Smart Campus Management System will include:

- User Authentication and Login Facility
- Department Management Facility
- Course and Module Management Facility
- Student/Faculty Information Management Facility
- Attendance Facility
- Notice Facility

2.3 Bibliometric Analysis:

Bibliometric analysis is a quantitative technique for evaluating research trends, patterns, and developments in a particular domain based on published literature. In terms of the Campus Manager system, bibliometric analysis helps in understanding the developments, importance, and impact of digital campus management systems and technology over a period of time.

Overview of Research Trends

Research on educational management systems, LMS, and campus automation systems has increased significantly over the last two decades. Research in this domain was initiated with a focus on automating individual systems such as student record systems and attendance systems. However, such systems have been limited in terms of scope and integration.

Research trends over the last few years have indicated a shift towards integrating multiple systems within a single platform. Research indicates that centralized systems have been more efficient in terms of operations, redundancy, and accessibility than traditional systems.

The integration of cloud computing, mobile technology, and artificial intelligence in educational systems has been a major area of discussion in recent literature.

Analysis of Key Research Areas

In the domain of campus management systems, the bibliometric analysis has shown the following key research areas:

1. Centralized Data Management

A significant number of studies have emphasized the need to manage the databases in a centralized manner. Research has shown that this will reduce the duplication of data and will provide greater consistency in managing the data.

2. Web-Based Systems

Research has shown that web-based applications are more efficient compared to traditional applications. This is because they are more accessible and are easier to maintain. The user can access the system anywhere, and this will provide greater flexibility to the users.

3. Role-Based Access Control

In the domain of educational institutes, security is one of the major issues. Research has shown that role-based access control is essential to provide greater security to the users. This will enable the users to access the relevant information based on their roles.

4. Automation of Processes

Another significant factor that has been emphasized in the recent studies is the need to automate the processes. Research has shown that the automated systems will reduce the chances of errors and will provide greater efficiency to the users.

5. User Experience and Interface Design

Recent research in the domain of campus management systems has also shown the need to provide greater user experience. This is because the user-friendly interface will provide greater usability to the users.

Publication Trends and Growth

From the bibliometric analysis, the growth in the number of publications related to campus management systems is evident. The growth is an indication of the increased importance of digital transformation in the education sector.

Trends:
Early 2000s – Emphasis on Digitization
2010-2015 – Emphasis on Web-Based Systems
2015-Present – Emphasis on Integration with AI, Cloud, and Analytics

The growth in the number of publications is an indication that educational institutions are keen on exploring the potential of advanced technologies to enhance their operations.

Technological Evolution

The evolution of technologies in campus management systems can be explained as follows:

Manual Systems → Computer-Based Systems → Web-Based Systems → Smart Systems

In the modern campus management system, the following technologies are integrated:

Cloud Technology – Scalability
Artificial Intelligence – Analytics
Mobile Technology – Accessibility

The Campus Manager project is an example of the technological evolution, as modern technologies are being incorporated, especially in the development of the system.

Gap Analysis

From the bibliometric analysis, the following gaps are evident in the existing campus management systems:

- Inadequate integration of all modules
- Inadequate scalability
- Inadequate user interface
- Inadequate security
- Inadequate complexity

Relevance to Campus Manager Project

The gaps and the growth in the number of publications are relevant to the development of the campus manager project. The gaps are an indication that the system being developed is expected to integrate all the modules, be scalable, have an adequate user interface, and be secure. The relevance is as follows:

From the analysis, the project is expected to integrate all the modules, as the gaps indicate inadequate integration in the existing systems. The project is expected to be scalable, as the gaps indicate inadequate scalability in the existing systems. The project is expected to have an adequate user interface, as the gaps indicate inadequate user interface in the existing systems. The project is expected to be secure, as the gaps indicate inadequate security in the existing systems. The project is expected to be less complex, as the gaps indicate inadequate complexity in the existing systems.

2.4 Review Summary

The review of existing literature and currently available campus management systems helps in developing a comprehensive understanding of the advantages, limitations, and gaps associated with traditional and new-age campus management systems. This review also plays a vital role in identifying the important requirements for developing a new and efficient system like Campus Manager.

Overview of Existing Systems

Existing campus management systems can be broadly classified into three categories:
Manual Systems, Standalone Digital Systems, and Integrated Web-Based Systems.

Manual Systems, also known as Paper-Based Systems, have been in use for a long time in educational institutions. Although they are easy to implement, they also possess many limitations, such as data redundancy, human errors, and difficulties in retrieving data, among others. As the student strength and activities increase, managing data manually has become inefficient and practically impossible.

Standalone digital systems were introduced to overcome some limitations associated with manual systems. This type of system uses computer-based applications to perform a variety of tasks, such as attendance management, student record management, and even examination management, among others. However, these systems often operate in isolation, leading to data

fragmentation, whereby data is distributed and scattered throughout different locations, and maintaining consistency and accuracy has become a serious problem.

Modern web-based systems have been developed to integrate various functions within a single system. These systems offer centralized data management and enable users to access data from different locations. However, existing web-based systems still have various challenges, including low scalability, complicated user interfaces, and inadequate security measures.

Strengths Identified in Existing Systems

From the literature review, various aspects have been identified regarding existing campus management systems, some of which are:

Automation of tasks for easy administration
Improved data organization through digital storage
Remote access to data
Role-based access control for data security
Integration of functions for easy coordination among departments

These aspects show the potential of digital systems in effective campus management.

Limitations Identified

Despite the various advantages offered by existing systems, some limitations have been identified, including:

One of the main limitations of existing systems is the lack of integration. Most systems have not been able to offer a single platform for all academic and administration functions, thus limiting effective campus management.

Another limitation is the user interface. Some systems have complicated interfaces, thus limiting effective campus management.

Scalability is another problem facing existing systems. Most systems have been developed without considering scalability, thus limiting effective data management.

Data security is another problem facing existing systems. In most systems, data security measures have not been adequately addressed, thus limiting effective data security.

Additionally, some systems have been expensive to maintain, thus limiting effective campus management for small and medium-sized institutions.

2.5 Problem Definition

The problem being addressed in the project is the inefficiency in the management of the school operations using traditional or outdated technology. Educational institutions are faced with challenges, including data inconsistency, lack of centralized data management, and inadequate communication among stakeholders.

The objectives of the project are to create a system that can:

- Automate the operations in the school
- Offer centralized data storage and management services
- Allow access to data in a secure manner
- Enhance communication among the stakeholders in the school

The solution must be able to handle large amounts of data in an efficient manner, be user-friendly, and be scalable.

2.6 Goals / Objectives

The main objective of the Campus Manager project is to develop an intelligent, efficient, and scalable system that facilitates the smooth functioning of educational institutions. The system is designed to replace conventional manual systems with an intelligent and centralized digital solution to improve productivity, accuracy, and accessibility of information.

Another important objective of the Campus Manager system is to enable the automation of administrative activities. Normally, administrative activities are performed manually. These activities include managing student information, tracking student attendance, managing courses, and managing other academic information. These activities are time-consuming when done manually. The system is designed to reduce manual effort, minimize errors, and enable administrative staff to focus on more important activities.

Another important objective of the Campus Manager system is to develop an intelligent and centralized solution for managing information. Normally, educational institutions have information scattered across different departments. Managing information is challenging when it is scattered. The Campus Manager system is designed to store information in a centralized manner. Information is stored in a unified database to ensure consistency. This will enable better organization of information and faster access to information when required.

The system also aims to improve data accuracy and accessibility. This means that manual handling of data, which often results in inaccurate and outdated information, is avoided in this system. The use of a structured database and automated handling of data helps to improve accuracy and accessibility of data, ensuring that data remains accurate and up-to-date and can be accessed by authorized users.

Improving communication among different stakeholders in a university or college is another important objective of this project. Communication among different stakeholders, such as students, teachers, and management, plays a vital role in ensuring proper functioning and management of a university or college. The system also aims to improve communication

through features such as notices, announcements, and updates, ensuring that all relevant information reaches its target audience in a proper and organized manner.

Security of information in a system is also a critical aspect, and therefore, this system aims to improve data security through features such as authentication and access control. Different users, such as management, teachers, and students, have different levels of access to information and data, and therefore, they can only access information relevant to their roles and duties, ensuring security and confidentiality of such information.

Another important objective of this project is to improve the interface and make it more user-friendly, allowing users to interact with the system easily and effectively. The interface of this system has been designed to be simple, clear, and easy to use, and users with little knowledge and expertise in handling such a system can also use it without much difficulty.

Lastly, the system has been designed to be scalable and flexible, such that it can accommodate future needs. This means that when educational institutions are growing, the system can also accommodate an increased number of users, courses, and records. Further, new features such as a mobile app, analytics, and additional modules can be incorporated in the future without having to make any changes to the current system.

In conclusion, the goals and objectives of the Campus Manager project revolve around efficiency, accuracy, communication, security, and scalability, among others. This provides a foundation for a system, and its development process aims to meet the needs of educational institutions in a real-world scenario.

2.7 Justification of Technology Selection

The choice of technologies, i.e., appropriate technologies, plays a vital role in the development of any software system. For the Campus Manager project, various modern technologies have been selected, which will ensure better performance, scalability, flexibility, and maintenance of the system. The system has been designed using a full-stack approach, where both frontend and backend technologies work together in harmony to deliver a powerful application.

Frontend Technology Selection

The frontend part of the Campus Manager system has been built using HTML, CSS, and JavaScript, which are considered the building blocks of modern web development. HTML has been used to structure the content of the web pages, CSS has been used to design and style the user interface, and JavaScript has been used to provide dynamic behavior to the application.

To provide better user experience and performance, modern technologies such as React.js can be employed. It will allow us to build modular applications, making the system more scalable and maintainable. It will also help in improving performance, where only required components will be updated instead of loading the entire webpage, thus providing better performance and a better user experience.

The use of these frontend technologies also helps in ensuring that the system developed is responsive, interactive, and accessible from different devices and screen sizes. Compared to desktop applications, web interfaces are much more flexible and easy to use.

Backend Technology Selection

The backend part of the Campus Manager system has been developed using Node.js and Express.js, which are leading technologies in developing scalable web applications. Node.js is a runtime environment that allows JavaScript to be executed on the server-side. It has a non-blocking, event-driven model, making it a very efficient platform for handling multiple requests concurrently.

Express.js is a lightweight web framework built on top of Node.js, allowing developers to build server-side applications more efficiently and effectively. It provides features such as routing, middleware, and API development, among others, to make developing server-side applications much simpler and more efficient.

The choice of using Node.js and Express.js for developing the backend part of the Campus Manager system has been justified in terms of its ability to use a single programming language, JavaScript, for both frontend and backend development, ensuring much simpler development and a shorter time to market. The choice has also been justified in terms of its ability to handle large volumes of data and its suitability for developing real-time applications.

Database Technology Selection

For the data storage, the Campus Manager system is using MongoDB as its database technology. MongoDB is a NoSQL database system. Unlike relational databases, MongoDB stores data in an unstructured form, called documents, which are essentially JSON-like data structures.

The use of MongoDB as the data storage technology provides the following advantages:

Flexible data schema

Improved data retrieval

Improved data scalability

Improved data handling, especially large data sets

Campus data, such as student data, course data, and attendance data, is often unstructured and varies in complexity. MongoDB's flexible data schema is highly suitable for the project, especially because MongoDB is also very scalable and integrates well with the other technologies being used, namely Node.js.

Comparison with Traditional Technologies

Traditional technologies, such as PHP, Java Servlets, and relational databases, are often used in traditional application systems. While they are reliable technologies, they are often associated with increased complexity and may not be as scalable and flexible as modern technologies.

In contrast, the technologies being used in the Campus Manager project are more efficient and

scalable, as they are modern technologies and are being compared with traditional technologies. The technologies being used are:

Improved development efficiency

Improved system performance through the use of asynchronous programming

Improved system scalability

Improved user interface through the use of dynamic technologies

Advantages of Selected Technologies

Scalability: The system is scalable and can handle an increased number of users and data without significant performance issues

Performance: Non-blocking architecture provides faster processing and increased response times

Flexibility: The system is flexible, and adding new features in the future is easy

Maintainability: The system is maintainable, and debugging is easy

Accessibility:

The system is accessible, as the system is web-based and can be accessed from any device

Community Support:

The technologies have large communities, and support is readily available

Security Considerations

Security is an important factor while choosing the appropriate technologies. The technologies we have chosen are appropriate to implement security features in the system. The system can be designed to have features such as secure password encryption, etc.

In addition, the technologies are secure against the following security threats:

- SQL Injection
- Cross-Site Scripting (XSS)
- Cross-Site Request Forgery (CSRF)

This ensures that the system is secure and the confidentiality and integrity of the data are maintained.

Future Scalability and Integration

Another important factor is the ability to scale the system in the future. The system is easily scalable and can have the following features in the future:

- Integration with mobile application
- Integration with cloud
- Integration with AI-based analytics
- Integration with real-time notifications

CHAPTER 3: SYSTEM ANALYSIS

3.1 Existing System Analysis

Analysis of the Existing System

Analysis of the existing system is an important part of understanding the current systems used by educational institutions to manage their activities. Analysis of the current systems used in educational institutions will provide insights into the strengths and limitations of traditional systems and will also help in understanding the areas that need to be improved.

In most educational institutions, the current system is manual, partially computerized, or based on standalone systems. Each of these systems has its advantages and limitations, and these limitations will impact the efficiency of the systems used in educational institutions.

Manual System

The manual system is one of the most traditional systems used in most educational institutions. Under this system, all the records relating to students, teachers, attendance, courses, and examinations are maintained manually.

Though the manual system is simple and does not require any technical knowledge, it has several disadvantages. One of the disadvantages is that managing large amounts of information manually is time-consuming. As the number of students and courses increases, managing the information becomes more complex.

Another disadvantage is that manual systems are more prone to human errors. There is a high possibility of errors occurring while entering the data into the registers. Moreover, the chances of losing the records are also high. Also, retrieving specific information from the manual records is time-consuming.

Another disadvantage of using manual systems is that they are more prone to unauthorized access. Anyone with access to the premises will have access to the information. Moreover, there are no specific guidelines for maintaining the back-up of the records, and this will result in the loss of information.

Partially Computerized System

In order to overcome the limitations of the manual system, many institutions have adopted the partially computerized system. In this system, some operations are performed using the computer, while other operations are performed manually.

Despite the fact that the partially computerized system reduces the amount of manual work, the system is still not an ideal solution. The main problem with the system is the data, as the data is inconsistent and duplicated. For example, the student data is maintained in one system, while the attendance data is maintained in another system.

In addition, the system lacks integration, and the data cannot be easily shared among the

different modules. Due to the lack of integration, the system is not efficient, and the chances of errors are high. The user may have to switch between the different applications to perform the different operations, and this reduces the productivity.

Standalone Digital Systems

Standalone digital systems are the computer applications developed to perform specific functions, such as maintaining student records, attendance, and examination results. The standalone system is more advanced compared to the other two systems, as the system provides high accuracy and efficiency.

Despite the fact that the standalone system is more advanced compared to the other two systems, the system is still not an ideal solution, as the system is characterized by a number of limitations. The first limitation is the lack of integration, as the system is developed to perform specific functions only and does not allow the integration of the other operations. Due to the lack of integration, the system is not efficient, as the data is managed in pieces.

In addition, the system lacks the feature that allows the user to access the data remotely, as the system is developed to perform operations on the local computer only. The system lacks the feature that allows the user to access the data remotely, and the user is restricted from accessing the data from outside the campus.

Furthermore, the system is characterized by the complexity and high cost, as the system requires the user to maintain multiple standalone systems.

Challenges in Existing Systems

The analysis of the existing system reveals the following challenges faced by the educational institution:

Inefficiency: The existing system is manual, and hence much time and effort are required

Data Redundancy: The data is duplicated and exists in multiple forms

Lack of Integration: The existing system is not integrated

Inaccessibility: The data is not accessible remotely

Poor Communication:The existing system lacks proper communication

Security: The data is not properly secured

Scalability: The system is not scalable, and hence data and users are not handled properly

All these challenges affect the educational institution as a whole and make it difficult to manage the operations efficiently.

Impact on the Stakeholders

The challenges faced by the existing system have a direct impact on the following stakeholders:

Administrators: The administrators are facing difficulties in managing the data and generating

reports

Faculty: The faculty is spending much time on administrative tasks instead of teaching

Students: The students are facing difficulties in accessing the data, such as results and notices

This reduces the productivity and hence the quality of the services provided by the educational institution.

Need for Improvement

From the analysis, the need to develop an improved system is very clear, as the existing system is not sufficient to cater to the needs of the educational institution. There is a need to develop a system that can integrate all the functionalities into a single system, provide data management, and ensure real-time access, as well as enhance the communication and security of the data.

3.2 Limitations of Existing System

The analysis of the existing system in the educational institution reveals several limitations, which are critical in nature, affecting the efficiency, reliability, and performance of the operations in the institution. Understanding these limitations is essential in justifying the need for an advanced system like the Campus Manager.

1. Heavy Dependence on Manual Processes

One of the most critical limitations of the existing system is the heavy dependence on manual processes. In the existing system, several operations, including the maintenance of student records, attendance records, course information, and administrative data, are handled manually.

Manual processes are highly time-consuming, particularly when handling large amounts of data. Moreover, the processes are repetitive, which results in delays in completing critical operations in the institution.

2. Data Redundancy and Inconsistency

Another limitation of the existing system is the presence of data redundancy and inconsistency in the operations of the institution.

In the existing system, the data is stored in different locations, which results in data redundancy and inconsistency in the operations of the institution.

For instance, the data of students is maintained separately in the administrative office, the academic departments, and the examination section.

3. Lack of Centralized Data Management

However, existing systems often do not have a central platform for managing such information within an institution. The information tends to be distributed among different types of files,

registers, or even applications, making it hard to access and use such information efficiently.

Lack of a central platform for managing information leads to a variety of problems, such as difficulties in retrieving information, increased risks of losing such information, and a lack of coordination among different departments within an institution. This also makes it hard to generate reports or even use such information for analysis and decision-making processes.

4. Limited Accessibility of Information

In traditional and standalone systems, accessibility to information tends to be limited to a particular location or device. Information stored in physical files or even in a local system does not allow for accessibility from a different location or even a different device.

Such limitations affect different individuals, such as students, faculty, and even administrators, who might need to access information from different locations within an institution. Lack of accessibility to information in real time also makes such processes inefficient and forces individuals to delay such important processes, such as updating records or even sharing information with different individuals.

5. Poor Communication Between Stakeholders

Communication is a vital aspect for ensuring the smooth functioning of any educational institution. However, existing systems often do not incorporate proper communication processes, leading to a variety of problems and even delays in different processes and activities.

Such limitations affect different individuals, such as students, faculty, and even administrators, who might need to communicate with different departments within an institution. Lack of communication also makes different processes inefficient and forces individuals to delay such important processes, such as updating records or even sharing information with different individuals.

6. High Risk of Human Errors

Standalone and traditional systems often tend to incorporate manual processes for managing information, leading to a variety of problems, such as human errors, which might prove to be critical in different processes and activities, such as maintaining academic records and even determining results for different examinations.

Since the system does not have any automatic validation or error checking, it is extremely difficult to detect and rectify errors. This would affect the overall system itself.

7. Lack of Security and Data Protection

Security is a major concern for the existing system. The manual system is highly prone to loss or damage, while the standalone system may lack adequate security features such as authentication and authorization.

Student details, teacher details, and results need to be stored securely. The problem is that the existing system may lack adequate security measures, and this would affect the system itself.

8. Scalability Issues

Since educational institutions are growing, the number of students and the overall data would also be increasing. The existing system may lack adequate scalability, and this would affect the system itself.

The manual system would become extremely difficult to manage, while the standalone system may lack adequate scalability and may not allow the addition of new features. This would affect the overall system itself.

9. Difficulty in Data Retrieval and Reporting

Retrieving any specific information is extremely difficult for manual systems, while for standalone systems, retrieving information may require a lot of time and effort to coordinate and get the desired output for a particular task, such as generating reports for student performance, attendance, and other statistics.

10. High Maintenance Effort

The maintenance of the existing system, particularly when standalone applications are in use, can be challenging since each system has to be updated, maintained, and supported individually.

This, in turn, increases the overall cost of running the system. Moreover, the updates are done across several platforms, which can lead to several inconsistencies.

11. Lack of Real-Time Processing

The existing system may not support real-time data processing, which means the updates made in one area of the system may not be reflected in other areas.

Real-time processing is an integral feature of an efficient system, which ensures the users are able to access the most current data in the system.

12. Limited User Experience

Most existing systems are not designed with the user experience in mind, which can make it very difficult for users to interact with the system.

This is particularly challenging when the users are not tech-savvy, which can make it very difficult for them to use the system efficiently.

Overall Impact of Limitations

The above limitations will lead to the following:

- Reduced system efficiency
- Increased workload for the staff members
- Delays in the processing of information
- Poor decision-making
- Reduced user satisfaction

These are the challenges facing the current system, which make it very imperative to introduce a new, integrated, and efficient system in the organization.

3.3 Proposed System Overview (Expanded)

The limitations and challenges that are evident in the existing system are clear indications that a better and more efficient system is required to meet the demands and requirements of the educational institution. To this end, the Campus Manager – Smart Campus Management System is proposed as a comprehensive and efficient system that is aimed at ensuring that the academic and administrative activities are streamlined and automated within an educational institution.

The proposed system is aimed at providing a unified environment that ensures all campus activities are managed efficiently. Unlike traditional systems, where different applications are used to manage different activities, the Campus Manager system integrates different applications into a single system, ensuring that all activities are coordinated and that information flows smoothly among different stakeholders.

Centralized Architecture

The proposed system is also characterized by a centralized system, whereby all the information regarding students, faculties, departments, courses, and activities is stored in a single database. The use of a centralized system ensures that the problem of data redundancy is addressed, and that all the information is consistent and uniform.

The use of a centralized system also ensures that users are able to access the required information from any place, and that the system is efficient and convenient for use. The use of a unified system also ensures that there is better control over the data, and that the system is efficient and convenient for use. The use of a unified system also ensures that the required information is retrieved efficiently and conveniently.

Web-Based Platform

The Campus Manager system is a web-based system, and this means that users are able to use the system without the need to install any software on their computers. The system is a web-based system, and this means that users are able to use the system without the need to install any software on their computers.

The fact that the system is web-based means that updates and changes can be made easily without having to make changes to each device individually. The system is also able to provide

real-time data access, thus making it easier for the user to access data instantly.

Role-Based Access Control

The system's security is an important factor, and the proposed system meets this requirement through the implementation of role-based access control (RBAC) by the Campus Manager system.

The system is designed to accommodate multiple user roles, each with its own set of privileges. The roles are as follows:

Administrator: The system's administrator, with full control over the system, including user, department, course, and system settings

Faculty: The faculty member, responsible for managing academic data, including courses and student data

Student: The student, responsible for viewing academic data, course data, and updates

This helps ensure data security, prevents unauthorized access, and maintains the confidentiality of sensitive data.

Modular Design Approach

The proposed system is designed using the modular approach, where the system is divided into smaller modules, each responsible for its own function.

The system's major modules are as follows:

- Authentication Module: Manages user security and login
- Department Management Module: Manages department data
- Course Management Module: Manages course data
- Module Management: Manages course modules
- User Management Module: Manages student and faculty data

This helps make the system easier to develop, maintain, and expand, as new modules can be added later without interfering with the existing ones.

Automation of Processes

One of the main objectives of the proposed system is the automation of processes, as most processes are done manually, and the system will help automate these processes.

Tasks such as student record keeping, course details, and administrative tasks are handled automatically. This means that staff and faculty can focus on other important matters rather than wasting time on repetitive tasks.

Real-Time Data Processing

The Campus Manager system also supports real-time data processing. This ensures that any changes are instantly updated to the system. This is important for keeping records and ensuring that users are provided with timely information.

For example, if a course is updated or a new department is added, this is instantly reflected to all relevant users. This ensures that users are provided with timely information and that everything is transparent.

User-Friendly Interface

The system that is proposed is also user-friendly. The system is designed to ensure that the user experience is positive and that users are able to use the system without any difficulties.

Scalability and Flexibility

The system is also designed to ensure that it is scalable. This means that the system is able to accommodate an increasing number of users and data as the institution expands. The system is also flexible and can be adapted to future changes that may be required.

Some possible future changes that may be required include:

- Integration with a mobile application
- Cloud deployment
- AI-based analytics
- Online attendance system

Security Features

The system that is proposed also includes a number of security features that ensure that the system is secure and that users are provided with a high level of security. Some of the security features that are included are:

- Secure authentication
- Password encryption
- Role-based access
- Unauthorized access protection

A feasibility study is an essential part of system analysis, which checks whether the proposed system is feasible, viable, and beneficial enough to be implemented. It checks the viability of the proposed system from various angles, including technical, financial, and operational viability.

The main aim of conducting a feasibility study for the Campus Manager system is to ensure that the proposed system development and implementation process will be carried out smoothly without facing many difficulties.

3.4.1 Technical Feasibility

Technical feasibility checks whether the proposed system can be implemented using existing technologies, tools, and facilities.

The Campus Manager system is technically feasible because it uses various state-of-the-art technologies, tools, and facilities, including HTML, CSS, JavaScript, Node.js, Express.js, and MongoDB, which are commonly used in the development of many applications in the software industry. These technologies, tools, and facilities are popular, and many developers use them, which makes development easier.

The proposed system does not use any special hardware facilities, and there is no need to use special hardware facilities or high-end facilities, tools, and technologies, which makes the proposed system more feasible. The proposed system will use a web-based approach, which means users will be able to use the proposed system using common computers and facilities, including common computers, tools, and facilities, which are commonly found in educational institutions.

Another important aspect related to technical feasibility is system performance and scalability. The use of Node.js, a non-blocking and event-driven programming paradigm, allows the system to handle multiple requests efficiently. Similarly, MongoDB has shown its ability to store large volumes of data, which makes it a suitable choice for handling large volumes of data related to institutions.

In addition, the availability of development tools such as Visual Studio Code, version control tools, and testing tools makes development, testing, and maintenance of the system smooth and efficient.

Thus, considering the availability of technology, tools, and human expertise, the Campus Manager system can be said to be highly feasible from a technical perspective.

3.4.2 Economic Feasibility

Economic feasibility assesses the cost-effectiveness of a proposed system by measuring its benefits and costs.

The Campus Manager system has been found to be economically feasible, considering its benefits and costs. The use of open-source tools and technologies helps reduce costs, as there are no costs associated with software licenses. For instance, Node.js, MongoDB, and front-end frameworks are open-source tools, and developers can use them free of charge, thus reducing development costs significantly.

In addition, the hardware requirements for the system are minimal, and it can operate on existing hardware, thus reducing costs associated with additional hardware requirements.

In addition to reducing development costs, the Campus Manager system has many advantages related to cost benefits, such as reduced labor costs and errors, and improved efficiency, which helps reduce time spent on performing day-to-day activities efficiently. For instance, automated processes reduce errors and time spent on performing day-to-day activities, thus reducing costs associated with labor and errors.

In addition, the system has a central platform, and updates and changes can be made from a single point, thus reducing costs associated with maintaining multiple standalone systems.

Overall, the benefits of the Campus Manager system outweigh the costs, and therefore, it is an economical and cost-effective system for educational institutions.

3.4.3 Operational Feasibility

Operational feasibility deals with how well the proposed system can be accommodated within the existing environment of an organization and how well it can be utilized by the people within an organization.

The Campus Manager system is user-friendly and easy to operate. It is designed with a high level of user experience in mind. The user interface is simple, user-friendly, and easy to operate.

The Campus Manager system can handle multiple user roles such as administrators, faculty members, and students. It is easy to use because it has clearly defined roles for different users. It can be easily accommodated within an existing environment without much hassle. It requires minimal training for its users because it is easy to understand and operate.

Moreover, it can help improve communication and coordination between different stakeholders within an organization. It can be effectively utilized for this purpose because it allows for information sharing between different stakeholders.

Thus, it can be concluded that the Campus Manager system is operationally feasible.

3.5 Requirement Analysis

Requirement analysis is an essential part of system development. It involves analyzing the needs and expectations of the people who will be utilizing the system. It is an important part of system development because it helps in designing an effective system.

For the Campus Manager system, requirement analysis involves analyzing the needs of administrators, professors, and students. The requirements can be classified as functional and non-functional requirements.

3.5.1 Functional Requirements

Functional requirements refer to the functions or operations that the system must perform.

The Campus Manager system includes various functional operations or requirements, such as:

User Registration and Authentication:

The system must allow users to register and authenticate themselves.

Role-Based Access Control:

Different users, such as administrators, professors, and students, have different levels of access rights. The system must restrict access based on user roles.

Student and Faculty Management:

The system must allow for the storage of detailed information regarding students and professors.

Department and Course Management:

The system must allow administrators to manage departments and courses effectively.

Module Management:

The system must allow for the management of different modules of courses.

Data Storage and Retrieval:

The system must allow for data storage and efficient data retrieval.

Dashboard Display:

The system must allow for displaying information to users through a dashboard.

The functional requirements of the Campus Manager system ensure that all operations required for effective campus management are included in the system.

3.5.2 Non-Functional Requirements

Non-functional requirements refer to the quality characteristics of the system.

Performance:

The system must allow for efficient performance, enabling it to respond quickly and allow multiple users without any delay.

Security:

The system must be equipped with security measures such as user authentication, access control, and data encryption.

Scalability:

It must also have the ability to cope with an increasing number of users and data with an increase in the size of the institution.

Usability:

It must have an interface that is simple, intuitive, and easy to use for any type of user.

Reliability:

It must be able to perform without any errors and provide accurate information.

Maintainability:

It must be simple to update, modify, and maintain.

These are non-functional requirements that will ensure the efficiency, security, and reliability of the system in real-world scenarios.

3.6 System Users and Roles

Campus Manager is designed to have multiple users with different roles and responsibilities. Each user will use the system according to their role.

Administrator

The administrator is the primary user of the system. He/she has complete control over the entire functionality of the system. The responsibilities of the administrator are to manage the entire system. These responsibilities include:

- Managing departments and courses
- Managing user accounts
- Monitoring activities
- Maintaining integrity

The administrator plays an important role in ensuring that the system is running smoothly and efficiently.

Faculty

The faculty members are responsible for managing the academic activities of the institution. Their responsibilities include:

- Managing courses
- Managing academic information
- Interacting with students
- Viewing relevant information

They use the system to make the management of academic activities simple.

Student

Students are the users of the system. They use the system to access information related to their activities. Their responsibilities include:

- Viewing courses
- Viewing student information
- Viewing announcements

They use the system to get information about courses and to stay updated with the latest announcements.

Role-Based Permissions

Each user will have specific permissions according to their roles. These permissions will ensure that the user does not have unauthorized access to the system. Also, they will use the system according.

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

System architecture is the overall structure of the system, and the interaction between the components. System architecture is the blueprint for the development of the system, and the components must interact effectively.

The system architecture of the Campus Manager system is Three-Tier Architecture.

Three-Tier Architecture Overview

The system is composed of the following components:

- Presentation Layer (Frontend)
- Application Layer (Backend / Server)
- Data Layer (Database)

Each part is responsible for its own function and communicates effectively with other components.

System Architecture Diagram (Draw This in Your Report)

```
┌──────────────────────────────────────┐
│  Presentation Layer                  │
│ (HTML, CSS, JavaScript UI)           │
│ - Login Page                         │
│ - Dashboard                          │
│ - Departments                        │
│ - Courses                            │
└──────────────────────────────────────┘
                │ HTTP Requests
                ▼
┌──────────────────────────────────────┐
| Application Layer                    |
│ (Node.js / Express Server)           │
│ - Authentication Logic               │
│ - Business Logic                     │
│ - API Handling                       │
└──────────────────────────────────────┘
                │Database Queries
                ▼
┌──────────────────────────────────────┐
│      Data Layer                      │
│      (MongoDB)                       │
│ - Users                              │
│ - Students                           │
│ - Courses                            │
│ - Departments                        │
└──────────────────────────────────────┘
```

You can also draw this using:

Word Shapes
Draw.io
PowerPoint

1. Presentation Layer (Frontend)

The presentation layer is the topmost layer in our system and hence serves as an interface between the user and our system.

In Campus Manager, we use HTML, CSS, JavaScript, and optionally React.js to implement our frontend.

In this layer, we include all the pages related to our application, such as:

- Login page
- Dashboard
- Departments page
- Courses page
- Modules page

The main responsibilities of this layer are:

- Handling user inputs such as login credentials and form data
- Displaying processed data received from our server
- Handling navigation from one page to another
- Making sure our interface is responsive and user-friendly

We use HTTP protocols such as GET, POST, PUT, and DELETE to communicate with our backend.

2. Application Layer (Backend)

The application layer serves as a brain for our system, handling all the processes and interacting with our database.

We use Node.js and Express.js to implement our backend, providing a platform for handling multiple users' requests.

Key Responsibilities:

- Handling API requests
- Validating user data

- Handling authentication and authorization
- Handling business logic such as course creation and handling users

Example of Flow:

- Form Submit
- Request Sent to Server
- Validation
- Response Sent to Frontend

In the backend, it is assured that the operations are performed securely and in an efficient manner.

3. Data Layer (Database)

The data layer is used to store and manage all the data used in the system. MongoDB is used as the database in the Campus Manager System.

Stored Data Includes:

- User Data
- Student Data
- Faculty Data
- Departments
- Courses and Modules

Responsibilities:

- Storage of Data
- Retrieval of Data
- Updation and Deletion of Data
- Maintenance of Data Integrity

MongoDB is used because it allows flexible schema designs and is also good at handling large amounts of data.

Working of System Architecture (Step-by-Step)

How the interaction of layers works is described below:

- How the user will interact with the frontend
- How the frontend will send the request to the backend
- How the backend will process the request
- How the backend will communicate with the database
- How the database will return the result to the backend
- How the backend will return the result to the frontend
- How the frontend will display the result to the user

4.2 Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) is a diagrammatic representation that shows the flow of data through the system. The DFD shows the flow of data among the users, processes, and data storage. In the case of the Campus Manager system, the DFD is important in understanding the interaction between the users and the system, as well as the internal data processing.

DFD is categorized into levels:

Level 0 (Context Diagram) → High-level overview

Level 1 → Detailed internal processes

The Level 0 Data Flow Diagram (DFD), also known as the Context Diagram, provides a high-level overview of the entire Campus Manager system. It represents the system as a single process and illustrates how it interacts with external entities such as users.

The purpose of the Level 0 DFD is to define the system boundary, identify external entities, and show the flow of data between the system and its users. It does not provide internal details but focuses on the overall interaction of the system with its environment.

Concept of Context Diagram

In the context diagram:

- The entire system is represented as one process
- External users are shown as entities
- Data flow is represented using arrows
- No internal modules are shown

This makes it easy to understand the system at a glance.

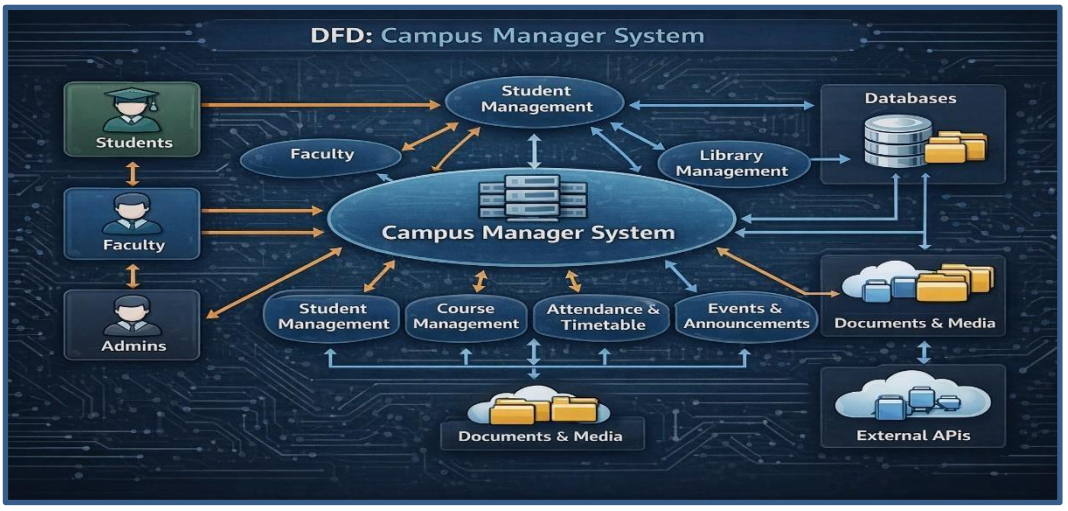


Figure: DFD Diagram

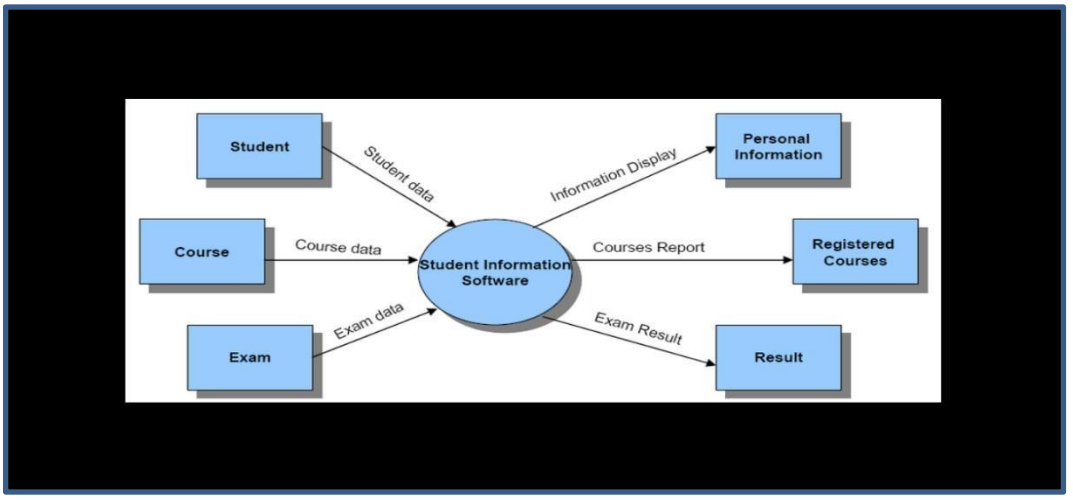
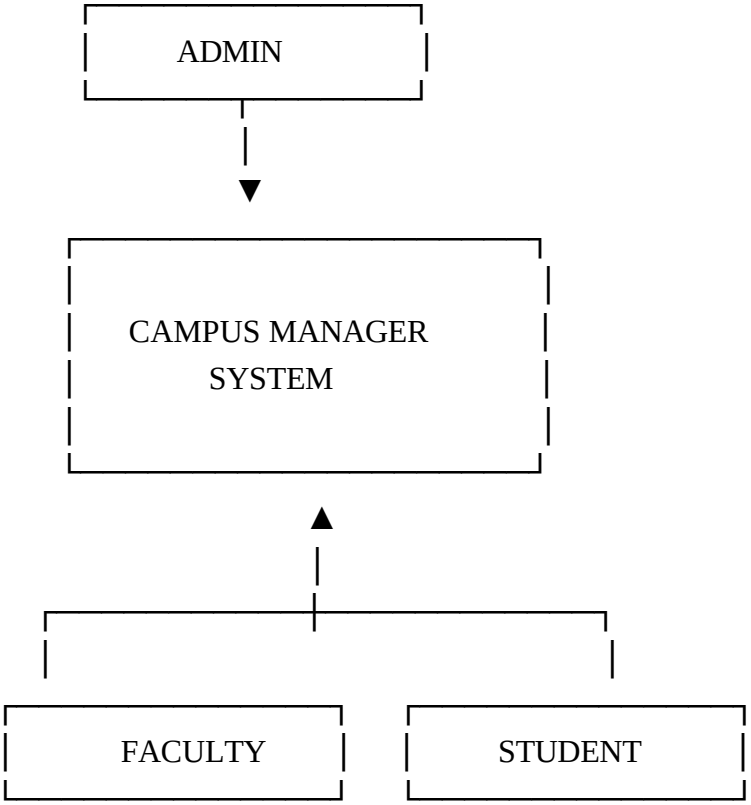


Figure-: DFD Diagram



External Entities

External entities are the users or systems that interact with the Campus Manager system.

1. Administrator

The administrator is the main controller of the system and performs operations such as:

- Managing departments
- Managing courses and modules
- Managing users (students and faculty)
- Monitoring system activities

The admin sends data such as:

- Course details
- Department information
- User data

And receives outputs such as:

- Reports
- Updated records
- Dashboard information

2. Faculty
Faculty members interact with the system for academic purposes. Their responsibilities include:

- Updating course-related data
- Managing academic content
- Accessing student information

They provide input such as:

- Course updates
- Academic data

They receive:

- Student information
- System notifications

3. Student
Students are the end users who access the system to view information.
They interact with the system by:

- Logging in
- Viewing courses and modules
- Accessing academic data

They provide input such as:

- Login credentials
- Queries

They receive:

- Course details
- Dashboard information

Main Process: Campus Manager System
The entire system is represented as a single process in Level 0 DFD.
Functions of the System:

- Accept user input
- Process requests
- Store and retrieve data
- Display output

The system acts as a central unit that connects all users and manages data flow.

Data Flow Explanation
Data flow represents how information moves between users and the system.
Input Data Flow

- Login credentials from users
- Course and department data from admin
- Academic updates from faculty

Output Data Flow

- Dashboard display
- Course information
- Notifications and updates

Working of Level 0 DFD
The working of the context diagram can be explained step-by-step:

1. The user (Admin/Faculty/Student) interacts with the system
2. The system receives input data
3. The system processes the data
4. The system generates output
5. Output is displayed to the user

This cycle continues for all interactions.

Characteristics of Level 0 DFD

- Represents the system as a single process
- Shows only external interactions
- Does not include internal modules
- Provides a high-level view
- Easy to understand and interpret

Importance of Level 0 DFD

The Level 0 DFD is important because:

- It defines the scope of the system
- It identifies system users
- It helps in understanding system boundaries
- It serves as a base for detailed diagrams (Level 1, Level 2)
- It simplifies communication between developers and stakeholder

4.2.2 Level 1 DFD

Level 1 DFD provides a detailed breakdown of system processes.
Main Processes:

1. User Authentication

2. Department Management
3. Course Management
4. Module Management
5. Data Storage

Flow Explanation:

- User logs in → authentication process
- Admin manages departments/courses
- Faculty updates academic data
- Data is stored in database
- System retrieves and displays data

This level shows how internal modules interact with each other.

4.3 Entity Relationship Diagram (ER Diagram**)**

The ER Diagram represents the relationship between different entities in the system.

Main Entities:

- User
- Student
- Faculty
- Department
- Course
- Module

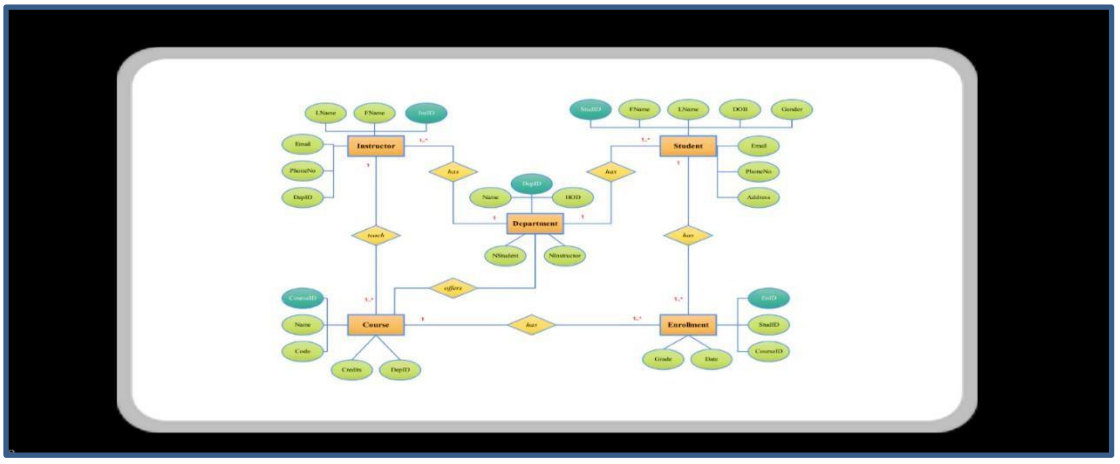
Relationships:

- A department has multiple courses
- A course has multiple modules
- A student enrolls in courses
- Faculty manages courses

Attributes Example:

- Student → ID, Name, Email
- Course → Course ID, Name
- Department → Dept ID, Name

The ER diagram helps in designing the database structure and ensures proper relationships between data entities.



4.4 Database Design

Database design defines how data is stored, organized, and managed.
The Campus Manager system uses a structured database to store information.

Main Tables / Collections:

1. User Table

- User ID
- Name
- Email
- Password
- Role

2. Student Table

- Student ID
- Name
- Course
- Department

3. Faculty Table

- Faculty ID
- Name
- Department

4. Department Table

- Department ID
- Department Name

5. Course Table

- Course ID
- Course Name
- Department ID

6. Module Table

- Module ID
- Course ID
- Module Name

The database is designed to ensure:

- Data consistency
- Efficient retrieval
- Minimal redundant

4.5 UML Diagrams (Expanded)

Unified Modeling Language (UML) is a standardized modeling language used to visualize, design, and document the structure and behavior of a system. In the

Campus Manager system, UML diagrams play a crucial role in representing system functionality, user interactions, and internal processes in a clear and structured manner.
UML diagrams help developers, designers, and stakeholders understand how the system operates without looking at the actual code. They improve communication, simplify complex system logic, and provide a blueprint for implementation.

Types of UML Diagrams Used

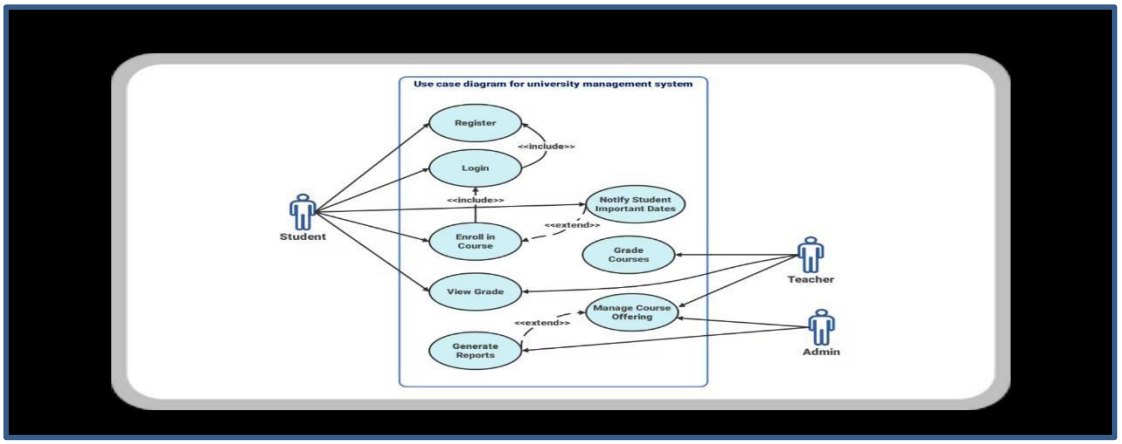
In the Campus Manager system, the following UML diagrams are used:

- Use Case Diagram → Shows user interactions
- Sequence Diagram → Shows step-by-step system flow

4.5.1 Use Case Diagram

A Use Case Diagram represents the interaction between users (actors) and the system. It shows what actions users can perform and how they interact with the system.
It focuses on:

- User roles
- System functionalities
- Interaction between users and system

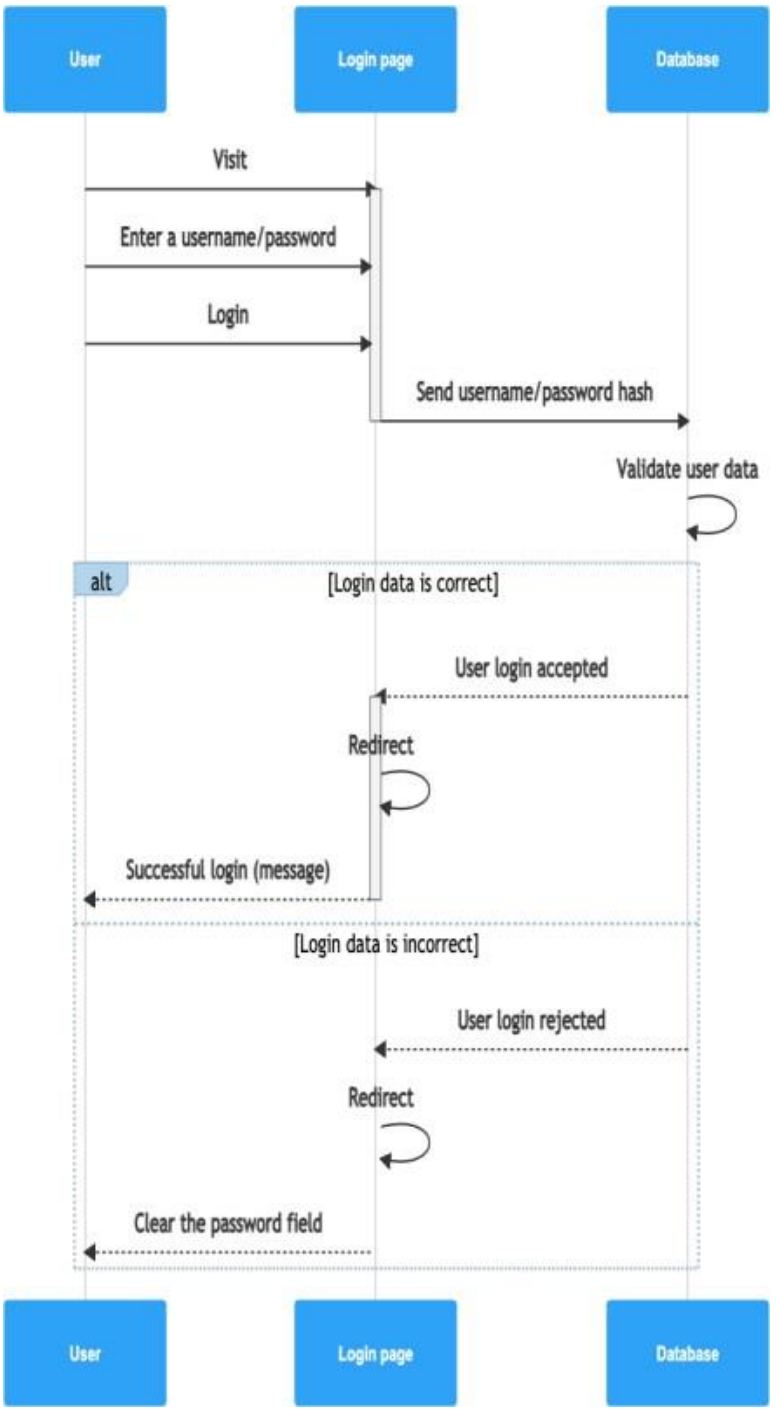


4.5.2 Sequence Diagram

A Sequence Diagram represents how different components of the system interact with each other in a sequence over time. It shows the flow of messages between objects step-by-step.
It focuses on:

- Order of operations
- Interaction between components
- Time-based execution

Sequence Diagram (Visual Reference)



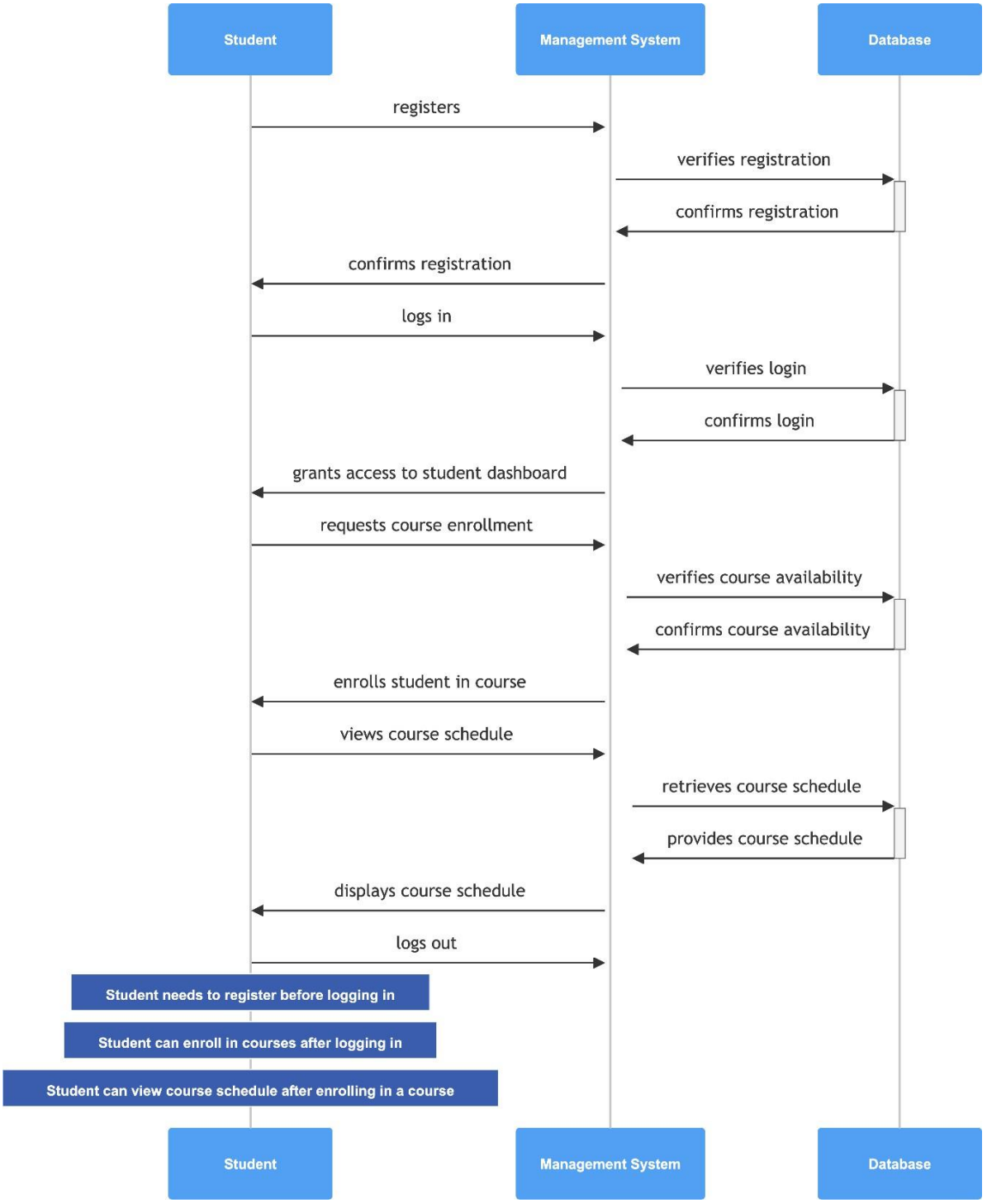


Figure : Sequence Diagram

Draw This Sequence Diagram (Login Example)
User → Frontend → Backend → Database

1. User enters login details
2. Frontend sends request to Backend
3. Backend checks data in Database
4. Database returns result
5. Backend sends response
6. Frontend displays dashboard

Detailed Flow Explanation

Step 1: User Input
User enters login credentials (username & password)

Step 2: Request to Server
Frontend sends HTTP request to backend server

Step 3: Validation
Backend validates user input

Step 4: Database Interaction
Database checks stored user credentials

Step 5: Response
Database sends result back to backend

Step 6: Output
Backend sends response to frontend → dashboard displayed.

4.6 User Interface (UI) Design

User Interface (UI) Design is one of the most important aspects of system development, as the interface determines how a system is used. The interface is one of the most important components that make a system easy to use, beautiful, and efficient enough to get the job done. In the Campus Manager system, particular emphasis is given to designing a beautiful, responsive, and easy-to-use interface to ensure a better user experience.

The UI is designed using modern web technologies such as HTML, CSS, JavaScript, and Bootstrap, making the system responsive to different devices such as computers, tablets, and mobile phones.

Objectives of UI Design

The main objectives of the user interface design are:

To ensure a simple and intuitive interface

To ensure a better user experience

To ensure easy navigation

To minimize errors and confusion

To ensure easy and fast access to important features

UI Design Principles Used

The Campus Manager system uses the following UI/UX design principles

1. Simplicity

The interface is simple and easy to use, and users can use the system without any confusion.

2. Consistency

The interface is consistent, and all pages are designed similarly, including the use of a consistent color scheme and design pattern.

3. Responsiveness

The interface is responsive, meaning the system is designed to work on different-sized screens and different devices

4. Visibility

The important features such as buttons, menus, and navigation are visible to the users**.**

5. Feedback

The system will provide feedback to users, such as displaying a message after a successful action or displaying an error messages**.**

UI Layout Diagram (Draw This)

```
+-----------------------------------------------------------------+

|              HEADER / NAVBAR                                    |

+- - - - - - - - - - - - - - - -+--------------------------------- - - - - - - -+

|                               |                                  |

|  SIDEBAR MENU                 |     MAIN CONTENT AREA            |

| - Dashboard                   |  - Data Display                  |

| - Departments                 |  - Forms                         |

| - Courses                     |  - Tables                        |

| - Modules                     |                                  |

|                               |                                  |

+---------------------------+-----------------------------------------+
```

Main UI Pages of Campus Manager

1. Home Page

The home page will be considered the entry point.

Features:

- Welcome message
- Navigation links
- Overview of system

Purpose:

- Provide overview
- Guide users through various sections

2. Login Page

The login page will be considered for user authentication.

Features:

- Username/email input
- Password input
- Login button

Design Focus:

- Simple and secure design
- Display error if invalid credentials

3. Dashboard

The dashboard will be considered the main interface.

Features:

- Overview of system data
- Quick links to various modules
- Navigation shortcuts

Purpose:

- Main interface for users
- Ease of use

4. Departments Page

Features:

- Display departments
- Add/edit/delete departments

UI Elements:

- Table view
- Buttons for actions

5. Courses Page

Features:

- Display list of courses
- Course management

UI Design:

- Organized interface
- Labels and titles clear

6. Modules Page

Features:

- Display list of course modules
- Manage modules

Purpose:

- Display structured course content

4.7 Design Constraints

Design constraints are the limitations, restrictions, and conditions that affect the system's design and development. Design constraints must be taken into consideration during the system design phase to ensure that the system is practical, efficient, and feasible.

In the case of the Campus Manager system, there are various constraints, restrictions, and conditions that affect the system's design and development. The following are the design constraints that affect the system's design and development. The system's design and development are affected by the following constraints, restrictions, and conditions.

1. Technical Constraints

Technical constraints are the restrictions, limitations, and conditions related to technology, tools, and system architecture.

The system is developed using various technologies, tools, and techniques, namely HTML, CSS, JavaScript, Node.js, MongoDB, and other related technologies. However, the system is affected by the following constraints, restrictions, and conditions.

The first constraint is the need for an internet connection, as the system is developed using the web. The system requires an internet connection to function properly. Without an internet connection, the system may not function properly.

Another constraint is the need to ensure that the system is compatible with the browsers that are being used, namely Google Chrome, Mozilla Firefox, and Microsoft Edge.

The system must be designed and developed to ensure that the system can handle multiple users simultaneously without slowing down.

For this, the system is designed to be lightweight and efficient, ensuring that it can run even on moderate hardware configurations.

3. Time Constraints

Time is an important factor that influences system development. The system development project has to be completed within a limited time frame, and this influences the system design and development process.

Due to time constraints, advanced features such as AI-based analytics or mobile applications may not be included in the system. The focus is on ensuring that the basic functionalities are implemented effectively within the time frame.

Proper planning and scheduling are required to ensure that all system development phases are completed within the time frame.

4. Security Constraints

Security is a major concern that influences system design. The Campus Manager system is designed to store sensitive information such as student and faculty details, and this information must be protected from unauthorized access.

The design constraints related to security are:

Implementation of secure authentication
Implementation of password encryption
Implementation of measures to prevent unauthorized access
Implementation of measures to ensure data privacy

Security and usability are two factors that are often conflicting, and this is a major challenge that influences system design and development.

CHAPTER 5: IMPLEMENTATION AND CODING

5.1 Development Environment

The development environment refers to the tools, technologies, and platforms employed in developing and implementing the Campus Manager system. An appropriate development environment helps in efficient coding, debugging, testing, and implementation of the system.

The Campus Manager system uses a full-stack approach in its development, incorporating frontend, backend, and database technologies.

Hardware Requirements

- Processor: Intel i3 and above
- RAM: 4GB and above
- Storage: 100GB and above

Software Requirements

- Operating System: Windows 10 and above
- Integrated Development Environment (IDE): Visual Studio Code
- Browser: Google Chrome/Firefox
- Version Control System: Git (optional)

Technologies Used

Frontend

- HTML → Structure of web pages
- CSS → Styling and Layout
- JavaScript → Adds interactivity to web pages
- Bootstrap → Responsive web design

Backend

- Node.js → Runtime environment for backend development
- Express.js → Backend framework

Database

- MongoDB → NoSQL database

The development environment helps in ensuring scalability, flexibility, and efficient system performance.

5.2 Frontend Implementation

The frontend plays a vital role in developing a Campus Manager system, and its implementation involves developing a responsive and interactive interface for users to interact with the system.

Key Features Implemented in Frontend
Responsive interface for different devices
Interactive interface elements
Form validation
Navigation system

Main Pages
Login Page
Dashboard Page
Departments Page
Courses Page
Modules Page

Workflow
User Interface → User interacts with the interface
User Interface → Data collected through forms and sent to backend APIs
User Interface → Response received from backend APIs and displayed

Advantages
Easy to use
Fast interaction
Attractive UI
5.3 Backend Implementation (Expanded)

The backend part handles the main part and business logic of the system. This part is implemented using Node.js and Express.js frameworks.

- Responsibilities of Backend
- Handling API
- Handling User Input
- Handling Authentication
- Handling Database
- API
- RESTful API
- GET
- Fetch
- POST
- Add
- PUT
- Update
- DELETE

- Remove
- Example
- User sends request
- Backend handles request
- Database
- Response sent

5.4 Database Implementation

The database part has been implemented using MongoDB, which stores data in JSON-like format.

- Collections Used
- Users
- Students
- Faculty
- Departments
- Courses
- Modules
- Database
- Insert
- Add
- Update
- Modify
- Delete
- Remove
- Find
- Retrieve
- Advantages
- Flexibility
- Fast
- Easy

5.5 Module Description

The system has been divided into modules for ease of management and organization.

5.5.1 Authentication Module

This part handles the login and security part of the system.

- Functions

- User Login
- Password Validation
- Role-Based
- Process
- User enters
- System validates
- Access granted/denied

5.5.2 Department Module

This part handles departments in the system.

- Functions
- Add Department
- Update Department
- Remove Department
- View Department

5.5.3 Course Module

This part handles course-related activities in the system.

- Functions
- Add Course
- Update Course
- Add Course to Department
- View Courses

5.5.4 Module Management

This part handles course modules in the system.

- Functions
- Add Module
- Add Module to Course
- Update Module
- Remove Module

5.6 Code Snippets:

Controllers:

assistantController:

```
  const asyncHandler = require('express-async-handler');
const { resolveOpenAiApiKey } = require('../utils/openAiKeyResolver');

const extractResponseText = (payload) => {
  if (payload && typeof payload.output_text === 'string' && payload.output_text.trim()) {
    return payload.output_text.trim();
  }

  if (!payload || !Array.isArray(payload.output)) return '';

  const parts = [];
  payload.output.forEach((item) => {
    if (!item || !Array.isArray(item.content)) return;
    item.content.forEach((chunk) => {
      if (!chunk) return;
      if (typeof chunk.text === 'string' && chunk.text.trim()) {
        parts.push(chunk.text.trim());
      }
    });
  });

  return parts.join('\n').trim();
};

const askAssistant = asyncHandler(async (req, res) => {
  const question = String(req.body && req.body.question ? req.body.question : '').trim();

  if (!question) {
    return res.status(400).json({ message: 'Question is required' });
  }

  if (question.length > 500) {
    return res.status(400).json({ message: 'Question is too long. Keep it under 500 characters.' });
  }

  const apiKey = resolveOpenAiApiKey();

  if (!apiKey) {
    return res.status(503).json({
      answer:
        'Assistant API key is not configured yet. Set OPENAI_API_KEY or use
OPENAI_API_KEY_ENC with OPENAI_KEY_ENC_SECRET.',
    });
  }

  const model = process.env.OPENAI_MODEL || 'gpt-4o-mini';
  const upstream = await fetch('https://api.openai.com/v1/responses', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${apiKey}`,
```

```
    },
    body: JSON.stringify({
      model,
      input: [
        {
          role: 'system',
          content: [
            {
              type: 'input_text',
              text:
                'You are a University CMS assistant. Respond only in English. Answer in plain text
only. Be short, practical, and focused on this CMS: departments, courses, modules, students,
enrollment, certificates, login, and portal.',
            },
          ],
        },
        {
          role: 'user',
          content: [{ type: 'input_text', text: question }],
        },
      ],
      temperature: 0.4,
      max_output_tokens: 250,
    }),
  });

  const data = await upstream.json().catch(() => ({}));
  if (!upstream.ok) {
    const errMsg =
      (data && data.error && data.error.message) || 'Assistant service is temporarily unavailable.
Please try again.';
    return res.status(502).json({ answer: errMsg });
  }

  const answer = extractResponseText(data);
  return res.json({
    answer:
      answer ||
      'I could not generate a response this time. Please ask again with a clearer question about the
CMS.',
  });
});

module.exports = { askAssistant };
```

authController:

```
const asyncHandler = require('express-async-handler');
const User = require('../models/User');
const wantsJson = require('../utils/wantsJson');
```

```
// Render login page
const showLogin = (req, res) => {
  if (req.session.user) return res.redirect('/');
  res.render('auth/login', { title: 'Admin Login' });
};

// Handle login
const login = asyncHandler(async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email: email.toLowerCase(), isActive: true });
  if (!user || !(await user.matchPassword(password))) {
    if (wantsJson(req)) {
      return res.status(401).json({ message: 'Invalid credentials' });
    }
    req.flash('error', 'Invalid email or password');
    return res.redirect('/login');
  }

  req.session.user = {
    _id: user._id,
    name: user.name,
    email: user.email,
    role: user.role,
    isActive: user.isActive,
  };
  if (wantsJson(req)) {
    return res.json({ message: 'Logged in', user: req.session.user });
  }
  req.flash('success', 'Logged in successfully');
  res.redirect('/');
});

// Handle logout
const logout = (req, res) => {
  req.session.destroy(() => {
  if (wantsJson(req)) {
      return res.json({ message: 'Logged out' });
    }
    res.redirect('/login');
  });
};

module.exports = { showLogin, login, logout };



    const asyncHandler = require('express-async-handler');
const PDFDocument = require('pdfkit');
const fs = require('fs');
const path = require('path');
const Certificate = require('../models/Certificate');
const Student = require('../models/Student');
```

```
const Course = require('../models/Course');
const Enrollment = require('../models/Enrollment');
const wantsJson = require('../utils/wantsJson');

const resolveLogoPath = () => {
  const fullPath = path.join(__dirname, '..', 'public', 'images', 'university-logo.png');
  if (fs.existsSync(fullPath)) {
    return fullPath;
  }
  return null;
};

const buildCertificateNumber = () => {
  const stamp = new Date().toISOString().slice(0, 10).replace(/-/g, '');
  const rand = Math.random().toString(36).slice(2, 8).toUpperCase();
  return `CERT-${stamp}-${rand}`;
};

const getStudentForSessionUser = async (sessionUser) => {
  const userId = sessionUser && sessionUser._id;
  const userEmail = sessionUser && sessionUser.email ?
String(sessionUser.email).toLowerCase() : '';
  if (!userId && !userEmail) return null;

  let student = null;
  if (userId) {
    student = await Student.findOne({ user: userId });
  }

  if (!student && userEmail) {
    student = await Student.findOne({ email: userEmail });
    if (student && !student.user && userId) {
      student.user = userId;
      await student.save();
    }
  }

  return student;
};

const streamCertificatePdf = (res, certificate) => {
  const studentName = certificate.student && certificate.student.name ? certificate.student.name
: 'Student';
  const rollNumber = certificate.student && certificate.student.rollNumber ?
certificate.student.rollNumber : '-';
  const courseName = certificate.course && certificate.course.name ? certificate.course.name :
'Course';
  const courseCode = certificate.course && certificate.course.code ? certificate.course.code : '-';
  const issueDate = new Date(certificate.issueDate).toLocaleDateString('en-GB');
  const issuedBy = certificate.issuedBy && certificate.issuedBy.name ?
certificate.issuedBy.name : 'Admin';
```

```
  const doc = new PDFDocument({ size: 'A4', margin: 50 });
  const filename = `${certificate.certificateNumber}.pdf`;
  const pageWidth = doc.page.width;
  const pageHeight = doc.page.height;
  const centerX = pageWidth / 2;
  const logoPath = resolveLogoPath();
  const logoSize = 34;
  const logoY = 71;

  res.setHeader('Content-Type', 'application/pdf');
  res.setHeader('Content-Disposition', `attachment; filename="${filename}"`);

  doc.pipe(res);

  // Outer and inner frame
  doc.rect(28, 28, pageWidth - 56, pageHeight - 56).lineWidth(2).stroke('#0f4c81');
  doc.rect(40, 40, pageWidth - 80, pageHeight - 80).lineWidth(1).stroke('#b38b2d');

  // Header stripe
  doc.roundedRect(65, 62, pageWidth - 130, 52, 8).fill('#0f4c81');

  // University logo (auto-load when image is available); otherwise draw a clean crest fallback.
  if (logoPath) {
    doc.image(logoPath, 78, logoY, { fit: [logoSize, logoSize], align: 'center', valign: 'center' });
    doc.image(logoPath, pageWidth - (78 + logoSize), logoY, {
      fit: [logoSize, logoSize],
      align: 'center',
      valign: 'center',
    });
  } else {
    doc.circle(95, 88, 16).lineWidth(1.2).fillAndStroke('#f8fafc', '#b38b2d');
    doc.circle(pageWidth - 95, 88, 16).lineWidth(1.2).fillAndStroke('#f8fafc', '#b38b2d');
    doc.fontSize(10).fillColor('#0f4c81').text('U', 90.7, 83.3);
    doc.fontSize(10).fillColor('#0f4c81').text('U', pageWidth - 99.3, 83.3);
  }

  doc
    .fillColor('#f8fafc')
    .fontSize(13)
    .text('UNIVERSITY COURSE MANAGEMENT SYSTEM', 65, 80, {
      width: pageWidth - 130,
      align: 'center',
    });

  // Accent line below header
  doc.moveTo(95, 130).lineTo(pageWidth - 95, 130).lineWidth(1.5).stroke('#b38b2d');

  // Badge
  doc.circle(centerX, 172, 26).lineWidth(2).stroke('#b38b2d');
  doc.fontSize(9).fillColor('#0f4c81').text('AWARD', centerX - 18, 166, { width: 36, align:
'center' });
```

```
doc
  .fillColor('#111827')
  .fontSize(34)
  .text('CERTIFICATE', 0, 212, { align: 'center' });

doc
  .fontSize(13)
  .fillColor('#374151')
  .text(certificate.title || 'Course Completion Certificate', 0, 252, { align: 'center' });

doc
  .fontSize(12)
  .fillColor('#334155')
  .text('This is proudly presented to', 0, 296, { align: 'center' });

doc
  .fontSize(28)
  .fillColor('#0b1f3a')
  .text(studentName, 85, 328, {
    width: pageWidth - 170,
    align: 'center',
    underline: true,
  });

doc
  .fontSize(11)
  .fillColor('#475569')
  .text(`Roll Number: ${rollNumber}`, 0, 374, { align: 'center' });

doc
  .fontSize(12)
  .fillColor('#334155')
  .text('for successful completion of', 0, 408, { align: 'center' });

doc
  .fontSize(19)
  .fillColor('#0f4c81')
  .text(`${courseName} (${courseCode})`, 70, 438, {
    width: pageWidth - 140,
    align: 'center',
  });

// Details panel
doc.roundedRect(85, 494, pageWidth - 170, 88, 6).lineWidth(1).stroke('#cbd5e1');
doc
  .fontSize(11)
  .fillColor('#1f2937')
  .text(`Certificate Number: ${certificate.certificateNumber}`, 102, 516, {
    width: pageWidth - 204,
    align: 'left',
  });
doc
```

```
    .fontSize(11)
    .text(`Issue Date: ${issueDate}`, 102, 536, {
      width: pageWidth - 204,
      align: 'left',
    });
  doc
    .fontSize(11)
    .text(`Issued By: ${issuedBy}`, 102, 556, {
      width: pageWidth - 204,
      align: 'left',
    });

  if (certificate.remarks) {
    doc
      .fontSize(10)
      .fillColor('#4b5563')
      .text(`Remarks: ${certificate.remarks}`, 85, 596, {
        width: pageWidth - 170,
        align: 'center',
      });
  }

  // Signature area
  doc.moveTo(120, 700).lineTo(280, 700).lineWidth(1).stroke('#64748b');
doc.moveTo(pageWidth - 280, 700).lineTo(pageWidth - 120,
700).lineWidth(1).stroke('#64748b');
  doc.fontSize(10).fillColor('#334155').text('Registrar', 120, 706, { width: 160, align: 'center' });
  doc.fontSize(10).fillColor('#334155').text('Authorized Signature', pageWidth - 280, 706, {
  width: 160,
    align: 'center',
  });

  // Footer line
  doc
    .fontSize(9)
    .fillColor('#64748b')
    .text('This certificate is system-generated and digitally verifiable.', 0, pageHeight - 62, { align:
'center' });

  doc.end();
};

const issueCertificate = asyncHandler(async (req, res) => {
  const { courseId, title, remarks } = req.body;
  const studentId = req.params.id;

  const student = await Student.findById(studentId).populate('course', 'name code');
  if (!student) {
    if (wantsJson(req)) return res.status(404).json({ message: 'Student not found' });
    req.flash('error', 'Student not found');
    return res.redirect('/students');
  }
```

```
  const course = await Course.findById(courseId || student.course?._id).select('name code');
  if (!course) {
    if (wantsJson(req)) return res.status(400).json({ message: 'Valid course is required' });
    req.flash('error', 'Valid course is required');
    return res.redirect(`/students/${student._id}`);
  }

  const allowedCourseIds = new Set();
  if (student.course) allowedCourseIds.add(String(student.course._id));
  if (student.user) {
    const enrollments = await Enrollment.find({ user: student.user, status: 'enrolled'
}).select('course');
    enrollments.forEach((entry) => allowedCourseIds.add(String(entry.course)));
  }

  if (!allowedCourseIds.has(String(course._id))) {
    if (wantsJson(req)) {
      return res.status(400).json({ message: 'Student is not enrolled in the selected course' });
    }
    req.flash('error', 'Student is not enrolled in the selected course');
    return res.redirect(`/students/${student._id}`);
  }

  const exists = await Certificate.findOne({ student: student._id, course: course._id });
  if (exists) {
    if (wantsJson(req)) return res.status(400).json({ message: 'Certificate already issued for this
course' });
    req.flash('error', 'Certificate already issued for this course');
    return res.redirect(`/students/${student._id}`);
  }

  const certificate = await Certificate.create({
    certificateNumber: buildCertificateNumber(),
    student: student._id,
    course: course._id,
    title: title && String(title).trim() ? String(title).trim() : 'Course Completion Certificate',
    remarks: remarks || '',
    issueDate: new Date(),
    issuedBy: req.session.user._id,
  });

  if (wantsJson(req)) return res.status(201).json(certificate);
  req.flash('success', 'Certificate issued successfully');
  res.redirect(`/students/${student._id}`);
});

const getMyCertificates = asyncHandler(async (req, res) => {
  const student = await getStudentForSessionUser(req.session.user);
  if (!student) {
    if (wantsJson(req)) return res.status(404).json({ message: 'Student profile not found' });
    req.flash('error', 'Student profile not found');
```

```
    return res.redirect('/portal/my-courses');
  }

  const certificates = await Certificate.find({ student: student._id })
    .populate('course', 'name code')
    .populate('issuedBy', 'name')
    .sort({ issueDate: -1 });

  if (wantsJson(req)) return res.json(certificates);
  res.render('portal/certificates', { title: 'My Certificates', certificates, student });
});

const downloadMyCertificate = asyncHandler(async (req, res) => {
  const student = await getStudentForSessionUser(req.session.user);
  if (!student) {
    if (wantsJson(req)) return res.status(404).json({ message: 'Student profile not found' });
    req.flash('error', 'Student profile not found');
    return res.redirect('/portal/my-courses');
  }

  const certificate = await Certificate.findById(req.params.id)
    .populate('student', 'name rollNumber')
    .populate('course', 'name code')
    .populate('issuedBy', 'name');

  if (!certificate || String(certificate.student._id) !== String(student._id)) {
    if (wantsJson(req)) return res.status(404).json({ message: 'Certificate not found' });
    req.flash('error', 'Certificate not found');
    return res.redirect('/portal/certificates');
  }

  return streamCertificatePdf(res, certificate);
});

const downloadCertificateAdmin = asyncHandler(async (req, res) => {
  const certificate = await Certificate.findById(req.params.id)
    .populate('student', 'name rollNumber')
    .populate('course', 'name code')
    .populate('issuedBy', 'name');

  if (!certificate) {
    if (wantsJson(req)) return res.status(404).json({ message: 'Certificate not found' });
    req.flash('error', 'Certificate not found');
    return res.redirect('/students');
  }

  return streamCertificatePdf(res, certificate);
});

module.exports = {
  issueCertificate,
  getMyCertificates,
```

```
    downloadMyCertificate,
    downloadCertificateAdmin,
};
```

assistantRateLimit.js:

```
const buckets = new Map();

const WINDOW_MS = Number(process.env.ASSISTANT_RATE_LIMIT_WINDOW_MS ||
60_000);
const MAX_REQUESTS = Number(process.env.ASSISTANT_RATE_LIMIT_MAX || 8);

function getClientId(req) {
  const forwarded = req.headers['x-forwarded-for'];
  if (typeof forwarded === 'string' && forwarded.trim()) {
    return forwarded.split(',')[0].trim();
  }
  return req.ip || req.connection?.remoteAddress || 'unknown-client';
}

function pruneBucket(now) {
  for (const [key, value] of buckets.entries()) {
    if (now >= value.resetAt) {
    buckets.delete(key);
    }
  }
}

function assistantRateLimit(req, res, next) {
  const now = Date.now();
  pruneBucket(now);

  const clientId = getClientId(req);
  let bucket = buckets.get(clientId);

  if (!bucket || now >= bucket.resetAt) {
    bucket = { count: 0, resetAt: now + WINDOW_MS };
    buckets.set(clientId, bucket);
  }

  bucket.count += 1;
  const remaining = Math.max(0, MAX_REQUESTS - bucket.count);

  res.setHeader('X-RateLimit-Limit', String(MAX_REQUESTS));
  res.setHeader('X-RateLimit-Remaining', String(remaining));
  res.setHeader('X-RateLimit-Reset', String(Math.ceil(bucket.resetAt / 1000)));

  if (bucket.count > MAX_REQUESTS) {
    const retryAfter = Math.max(1, Math.ceil((bucket.resetAt - now) / 1000));
    res.setHeader('Retry-After', String(retryAfter));
    return res.status(429).json({
      answer: 'Too many assistant requests. Please wait a moment and try again.',
```

```
        });
      }

      return next();
    }

    module.exports = assistantRateLimit;
```

Style.css :

```
        /* Custom styles */
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 50%, #a8c0d8 100%);
    min-height: 100vh;
}

/* Hero Section */
.hero-section {
    position: relative;
    height: 100vh;
    overflow: hidden;
    display: flex;
    align-items: center;
    justify-content: center;
    color: white;
    text-align: center;
}

.video-container {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    z-index: -2;
}

.hero-video {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    object-fit: cover;
}

.hero-overlay {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
```

```
    background: linear-gradient(135deg, rgba(41, 128, 185, 0.8), rgba(142, 68, 173, 0.8), rgba(52, 73,
94, 0.9));
    z-index: -1;
}

.hero-content {
    position: relative;
    z-index: 1;
    max-width: 800px;
    padding: 0 20px;
}

.hero-title {
    font-size: 4rem;
    font-weight: 800;
    margin-bottom: 1.5rem;
    text-shadow: 3px 3px 6px rgba(0, 0, 0, 0.7);
    animation: fadeInUp 1.5s ease-out;
}

.hero-subtitle {
    font-size: 1.8rem;
    margin-bottom: 2.5rem;
    text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.7);
    animation: fadeInUp 1.5s ease-out 0.3s both;
}

.hero-buttons {
    animation: fadeInUp 1.5s ease-out 0.6s both;
}

.hero-buttons .btn {
    margin: 0 10px;
    padding: 15px 35px;
    font-size: 1.2rem;
    font-weight: 600;
    border-radius: 50px;
    transition: all 0.3s ease;
    text-transform: uppercase;
    letter-spacing: 1px;
}

.btn-hero-primary {
    background: linear-gradient(45deg, #FF6B6B, #4ECDC4);
    border: none;
    color: white;
}

.btn-hero-primary:hover {
    transform: translateY(-3px);
    box-shadow: 0 10px 25px rgba(255, 107, 107, 0.4);
}
```

```
.btn-hero-outline {
    border: 2px solid white;
    color: white;
    background: transparent;
}

.btn-hero-outline:hover {
    background: white;
    color: #2980b9;
    transform: translateY(-3px);
}

@keyframes fadeInUp {
    from {
        opacity: 0;
        transform: translateY(30px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

/* Floating Icons */
.floating-icons {
    position: fixed;
    top: 50%;
    transform: translateY(-50%);
    z-index: 1000;
}

.chat-icon {
    right: 30px;
}

.whatsapp-icon {
    left: 30px;
}

.floating-icon  {
    width:  60px;
    height: 60px;
    border-radius: 50%;
    display: flex;
    align-items: center;
    justify-content: center;
    color: white;
    font-size: 24px;
    text-decoration: none;
    margin: 15px 0;
    transition: all 0.3s ease;
```

```
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.2);
    animation: pulse 2s infinite;
    position: relative;
}

.floating-icon:hover {
    transform: scale(1.1);
    box-shadow: 0 6px 20px rgba(0, 0, 0, 0.3);
    text-decoration: none;
    color: white;
}

.chat-float {
    background: linear-gradient(45deg, #667eea, #764ba2);
    position: relative;
    overflow: visible;
}

.chatgpt-float {
    background: linear-gradient(135deg, #0f4c81, #00a67e);
    width: auto;
    height: 56px;
    border-radius: 999px;
    padding: 0 18px 0 10px;
    gap: 10px;
    font-size: 20px;
    animation: none;
}

.chatgpt-float i {
    width: 40px;
    height: 40px;
    border-radius: 50%;
    display: flex;
    align-items: center;
    justify-content: center;
    background: rgba(255, 255, 255, 0.18);
}

.chatgpt-float-text {
    color: #ffffff;
    font-size: 14px;
    font-weight: 700;
    letter-spacing: 0.2px;
    white-space: nowrap;
}

.chatgpt-float::before {
    display: none;
}

.chatgpt-float::after {
```

```
    display: none;
}

.chat-float::before {
    content: '';
    position: absolute;
    top: -5px;
    left: -5px;
    right: -5px;
    bottom: -5px;
    background: linear-gradient(45deg, #667eea, #764ba2, #c06c84, #feca57);
    border-radius: 50%;
    z-index: -1;
    animation: rotate 3s linear infinite;
    opacity: 0.7;
}

.chat-float::after {
    content: '';
    position: absolute;
    top: -2px;
    left: -2px;
    right: -2px;
    bottom: -2px;
    background: linear-gradient(45deg, #667eea, #764ba2);
    border-radius: 50%;
    z-index: -1;
    animation: pulse-ring 2s ease-out infinite;
}

.chat-bubble {
    position: absolute;
    top: -40px;
    right: 70px;
    background: white;
    color: #333;
    padding: 8px 15px;
    border-radius: 20px;
    font-size: 14px;
    font-weight: 600;
    white-space: nowrap;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.2);
    opacity: 0;
    transform: translateX(20px);
    transition: all 0.3s ease;
    pointer-events: none;
}

.chat-bubble::after {
    content: '';
    position: absolute;
    top: 50%;
```

```
    right: -8px;
    transform: translateY(-50%);
    width: 0;
    height: 0;
    border-left: 8px solid white;
    border-top: 8px solid transparent;
    border-bottom: 8px solid transparent;
}

.chat-icon:hover .chat-bubble {
    opacity: 1;
    transform: translateX(0);
}

.chat-icon:hover .chat-float i {
    animation: bounce 0.6s ease-in-out;
}

.whatsapp-float {
    background: linear-gradient(45deg, #25D366, #128C7E);
}

.call-float {
    background: linear-gradient(45deg, #FF6B6B, #FF8E53);
}

@keyframes pulse {
    0% {
        box-shadow: 0 0 0 0 rgba(102, 126, 234, 0.7);
    }
    70% {
        box-shadow: 0 0 0 10px rgba(102, 126, 234, 0);
    }
    100% {
        box-shadow: 0 0 0 0 rgba(102, 126, 234, 0);
    }
}

@keyframes rotate {
    0% {
        transform: rotate(0deg);
    }
    100% {
        transform: rotate(360deg);
    }
}

@keyframes pulse-ring {
    0% {
        transform: scale(1);
        opacity: 1;
    }
```

```
    100% {
        transform: scale(1.3);
        opacity: 0;
    }
}

@keyframes bounce {
    0%, 20%, 50%, 80%, 100% {
        transform: translateY(0);
    }
    40% {
        transform: translateY(-10px);
    }
    60% {
        transform: translateY(-5px);
    }
}

/* Login Modal Styles */
.login-modal {
    display: none;
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: rgba(0, 0, 0, 0.7);
    z-index: 9999;
    backdrop-filter: blur(5px);
}

.login-modal-content {
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
    background: white;
    border-radius: 20px;
    box-shadow: 0 20px 60px rgba(0, 0, 0, 0.3);
    max-width: 450px;
    width: 90%;
    overflow: hidden;
    animation: modalSlideIn 0.3s ease-out;
}

@keyframes modalSlideIn {
    from {
        opacity: 0;
        transform: translate(-50%, -60%);
    }
    to {
        opacity: 1;
```

```
        transform: translate(-50%, -50%);
    }
}

.login-modal-header {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
    padding: 1.5rem;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.login-modal-title {
    font-size: 1.5rem;
    font-weight: 700;
    margin: 0;
}

.login-modal-close {
    background: none;
    border: none;
    color: white;
    font-size: 1.5rem;
    cursor: pointer;
    padding: 0.5rem;
    border-radius: 50%;
    transition: all 0.3s ease;
}

.login-modal-close:hover {
    background: rgba(255, 255, 255, 0.2);
}

.login-modal-body {
    padding: 2rem;
}

.form-label {
    font-weight: 600;
    color: #2c3e50;
    margin-bottom: 0.5rem;
}

.btn-login-submit {
    background: linear-gradient(45deg, #667eea, #764ba2);
    color: white;
    border: none;
    padding: 1rem;
    border-radius: 10px;
    font-size: 1.1rem;
    font-weight: 600;
```

```
    transition: all 0.3s ease;
}

.btn-login-submit:hover {
    transform: translateY(-2px);
    box-shadow: 0 10px 25px rgba(102, 126, 234, 0.3);
}

.login-modal-footer {
    margin-top: 1.5rem;
    padding-top: 1.5rem;
    border-top: 1px solid #e9ecef;
    text-align: center;
}

.login-link {
    color: #667eea;
    text-decoration: none;
    font-weight: 600;
}

.login-link:hover {
    color: #764ba2;
    text-decoration: underline;
}

.btn-hero-login {
    background: linear-gradient(45deg, #28a745, #20c997);
    color: white;
    border: none;
    padding: 15px 30px;
    border-radius: 50px;
    font-size: 1.1rem;
    font-weight: 600;
    text-decoration: none;
    display: inline-block;
    transition: all 0.3s ease;
    cursor: pointer;
}

.btn-hero-login:hover {
    transform: translateY(-3px);
    box-shadow: 0 10px 25px rgba(40, 167, 69, 0.3);
    color: white;
}
.study-section {
    padding: 80px 0;
    position: relative;
    overflow: hidden;
}

.study-section-light {
```

```
    background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
}

.study-section-dark {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
}

.study-section-pattern {
    background:
        linear-gradient(135deg, rgba(102, 126, 234, 0.9), rgba(118, 75, 162, 0.9)),
        url('data:image/svg+xml,<svg xmlns="http://www.w3.org/2000/svg" width="100" height="100"
viewBox="0 0 100 100"><rect width="100" height="100" fill="%23ffffff" opacity="0.03"/><circle
cx="50" cy="50" r="40" fill="none" stroke="%23ffffff" stroke-width="0.5" opacity="0.1"/></svg>');
    color: white;
}

.section-title {
    font-size: 3rem;
    font-weight: 700;
    margin-bottom: 1rem;
    text-align: center;
    position: relative;
}

.section-subtitle {
    font-size: 1.2rem;
    text-align: center;
    margin-bottom: 3rem;
    opacity: 0.8;
}

.feature-card {
    background: white;
    border-radius: 15px;
    padding: 2rem;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);
    transition: all 0.3s ease;
    height: 100%;
    position: relative;
    overflow: hidden;
}

.feature-card::before {
    content: '';
    position: absolute;
    top: 0;
    left: 0;
    right: 0;
    height: 5px;
    background: linear-gradient(45deg, #667eea, #764ba2);
}
```

```
.feature-card:hover {
    transform: translateY(-10px);
    box-shadow: 0 20px 40px rgba(0, 0, 0, 0.15);
}

.feature-icon   {
    width:  80px;
    height: 80px;
    border-radius: 50%;
    background: linear-gradient(45deg, #667eea, #764ba2);
    display: flex;
    align-items: center;
    justify-content: center;
    margin: 0 auto 1.5rem;
    font-size: 2rem;
    color: white;
}

.stats-box {
    text-align: center;
    padding: 2rem;
    background: rgba(255, 255, 255, 0.1);
    border-radius: 15px;
    backdrop-filter: blur(10px);
    border: 1px solid rgba(255, 255, 255, 0.2);
    transition: all 0.3s ease;
}

.stats-box:hover {
    transform: scale(1.05);
    background: rgba(255, 255, 255, 0.15);
}

.stats-number {
    font-size: 3rem;
    font-weight: 700;
    margin-bottom: 0.5rem;
    background: linear-gradient(45deg, #FFD700, #FFA500);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    background-clip: text;
}

.testimonial-card {
    background: white;
    border-radius: 15px;
    padding: 2rem;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);
    margin-bottom: 2rem;
    position: relative;
}
```

```
.testimonial-card::before {
    content: "";
    position: absolute;
    top: -10px;
    left: 20px;
    font-size: 4rem;
    color:  #667eea;
    opacity: 0.3;
}

.testimonial-text {
    font-style: italic;
    margin-bottom: 1.5rem;
    font-size: 1.1rem;
    line-height: 1.6;
}

.testimonial-author {
    font-weight: 600;
    color: #667eea;
}

.program-card {
    background: white;
    border-radius: 15px;
    overflow: hidden;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);
    transition: all 0.3s ease;
    height: 100%;
}

.program-card:hover {
    transform: translateY(-5px);
    box-shadow: 0 15px 35px rgba(0, 0, 0, 0.15);
}

.program-image {
    height: 200px;
    background: linear-gradient(45deg, #667eea, #764ba2);
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 3rem;
    color: white;
}

.program-content {
    padding: 1.5rem;
}

.cta-section {
```

```
    background: linear-gradient(135deg, #FF6B6B 0%, #4ECDC4 100%);
    color: white;
    padding: 80px 0;
    text-align: center;
}

.cta-title {
    font-size: 2.5rem;
    font-weight: 700;
    margin-bottom: 1rem;
}

.cta-subtitle {
    font-size: 1.2rem;
    margin-bottom: 2rem;
    opacity: 0.9;
}

.btn-cta {
    background: white;
    color: #FF6B6B;
    padding: 15px 40px;
    border-radius:  50px;
    font-weight: 600;
    text-decoration: none;
    display: inline-block;
    transition: all 0.3s ease;
    text-transform: uppercase;
    letter-spacing: 1px;
}

.btn-cta:hover {
    transform: translateY(-3px);
    box-shadow: 0 10px 25px rgba(0, 0, 0, 0.2);
    color: #FF6B6B;
    text-decoration: none;
}

/* Enhanced Footer Styles */
.footer {
    background: linear-gradient(135deg, #1a1a2e 0%, #16213e 25%, #0f3460 50%, #533483 75%,
#c06c84 100%);
    color: white;
    border-top: 5px solid transparent;
    border-image: linear-gradient(45deg, #667eea, #764ba2, #c06c84, #feca57) 1;
    position: relative;
    overflow: hidden;
    padding-top: 80px !important;
}

.footer::before {
    content: '';
```

```
    position: absolute;
    top: 0;
    left: 0;
    right: 0;
    height: 100%;
    background:
        radial-gradient(circle at 20% 50%, rgba(102, 126, 234, 0.1) 0%, transparent 50%),
        radial-gradient(circle at 80% 50%, rgba(192, 108, 132, 0.1) 0%, transparent 50%),
        url('data:image/svg+xml,<svg xmlns="http://www.w3.org/2000/svg" width="200" height="200"
viewBox="0 0 200 200"><defs><pattern id="grid" width="20" height="20"
patternUnits="userSpaceOnUse"><path d="M 20 0 L 0 0 0 20" fill="none"
stroke="rgba(255,255,255,0.03)" stroke-width="1"/></pattern></defs><rect width="200"
height="200" fill="url(%23grid)"/><circle cx="50" cy="50" r="30" fill="none"
stroke="rgba(255,255,255,0.02)" stroke-width="0.5"/><circle cx="150" cy="150" r="40" fill="none"
stroke="rgba(255,255,255,0.02)" stroke-width="0.5"/></svg>');
    z-index: 1;
}

.footer-content {
    position: relative;
    z-index: 2;
}

.footer-section {
    margin-bottom: 2rem;
    animation: fadeInUp 0.8s ease-out;
}

.footer-title {
    font-size: 1.5rem;
    font-weight: 700;
    margin-bottom: 1.5rem;
    background: linear-gradient(45deg, #667eea, #c06c84);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    background-clip: text;
    position: relative;
    padding-bottom: 0.5rem;
}

.footer-title::after {
    content: '';
    position: absolute;
    bottom: 0;
    left: 0;
    width: 50px;
    height: 3px;
    background: linear-gradient(45deg, #667eea, #c06c84);
    border-radius: 2px;
}

.footer-description {
```

```
    color: rgba(255, 255, 255, 0.8);
    line-height: 1.6;
    margin-bottom: 1.5rem;
}

.footer-links {
    list-style: none;
    padding: 0;
}

.footer-links li {
    margin-bottom: 0.8rem;
}

.footer-links a {
    color: rgba(255, 255, 255, 0.7);
    text-decoration: none;
    transition: all 0.3s ease;
    display: inline-flex;
    align-items: center;
    padding: 0.3rem 0;
}

.footer-links a:hover {
    color: #667eea;
    transform: translateX(5px);
}

.footer-links i {
    margin-right: 0.5rem;
    font-size: 0.9rem;
}

.contact-info {
    list-style: none;
    padding: 0;
}

.contact-info li {
    margin-bottom: 1rem;
    color: rgba(255, 255, 255, 0.8);
    display: flex;
    align-items: center;
}

.contact-info i {
    margin-right: 1rem;
    color: #667eea;
    font-size: 1.1rem;
    width: 20px;
}
```

```
.social-icons {
    display: flex;
    gap: 1rem;
    margin-top: 1.5rem;
}

.social-icon {
    width: 45px;
    height: 45px;
    border-radius: 50%;
    background: rgba(255, 255, 255, 0.1);
    display: flex;
    align-items: center;
    justify-content: center;
    color: white;
    text-decoration: none;
    transition: all 0.3s ease;
    border: 1px solid rgba(255, 255, 255, 0.2);
    backdrop-filter: blur(10px);
}

.social-icon:hover {
    background: linear-gradient(45deg, #667eea, #764ba2);
    transform: translateY(-3px) scale(1.1);
    box-shadow: 0 10px 20px rgba(102, 126, 234, 0.3);
    color: white;
    text-decoration: none;
}

.footer-bottom {
    margin-top: 3rem;
    padding-top: 2rem;
    border-top: 1px solid rgba(255, 255, 255, 0.1);
    text-align: center;
    position: relative;
}

.footer-bottom::before {
    content: '';
    position: absolute;
    top: -1px;
    left: 50%;
    transform: translateX(-50%);
    width: 100px;
    height: 2px;
    background: linear-gradient(45deg, #667eea, #c06c84);
}

.footer-copyright {
    color: rgba(255, 255, 255, 0.6);
    font-size: 0.9rem;
}
```

```
.footer-heart {
    color: #e74c3c;
    animation: heartbeat 1.5s ease-in-out infinite;
}

@keyframes heartbeat {
    0%, 100% { transform: scale(1); }
    50% { transform: scale(1.1); }
}

.newsletter-form {
    margin-top: 1.5rem;
}

.newsletter-input {
    background: rgba(255, 255, 255, 0.1);
    border: 1px solid rgba(255, 255, 255, 0.2);
    border-radius: 25px;
    padding: 0.8rem 1.2rem;
    color: white;
    width: 100%;
    margin-bottom: 1rem;
    backdrop-filter: blur(10px);
    transition: all 0.3s ease;
}

.newsletter-input::placeholder {
    color: rgba(255, 255, 255, 0.5);
}

.newsletter-input:focus {
    outline: none;
    border-color: #667eea;
    background: rgba(255, 255, 255, 0.15);
}

.newsletter-btn {
    background: linear-gradient(45deg, #667eea, #764ba2);
    border: none;
    border-radius: 25px;
    padding: 0.8rem 2rem;
    color: white;
    font-weight: 600;
    cursor: pointer;
    transition: all 0.3s ease;
    width: 100%;
}

.newsletter-btn:hover {
    transform: translateY(-2px);
    box-shadow: 0 10px 20px rgba(102, 126, 234, 0.3);
```

```
}

.footer-badge {
    display: inline-block;
    background: linear-gradient(45deg, #feca57, #ff6b6b);
    color: #2c3e50;
    padding: 0.3rem 0.8rem;
    border-radius: 15px;
    font-size: 0.8rem;
    font-weight: 600;
    margin-top: 1rem;
}

/* Courses Page Styles */
.courses-hero {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
    padding: 80px 0 60px;
    margin-bottom: 3rem;
    position: relative;
    overflow: hidden;
}

.courses-hero::before {
    content: '';
    position: absolute;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background: url('data:image/svg+xml,<svg xmlns="http://www.w3.org/2000/svg" width="100"
height="100" viewBox="0 0 100 100"><circle cx="50" cy="50" r="30" fill="none"
stroke="rgba(255,255,255,0.1)" stroke-width="0.5"/><circle cx="20" cy="20" r="10"
fill="rgba(255,255,255,0.05)"/><circle cx="80" cy="80" r="15"
fill="rgba(255,255,255,0.05)"/></svg>');
    z-index: 1;
}

.courses-hero-content {
    position: relative;
    z-index: 2;
}

.courses-title {
    font-size: 3rem;
    font-weight: 700;
    margin-bottom: 1rem;
    text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.3);
}

.courses-subtitle {
    font-size: 1.3rem;
```

```
    opacity: 0.9;
    margin-bottom: 2rem;
}

.courses-stats  {
    display: flex;
    gap: 3rem;
    justify-content: center;
    flex-wrap: wrap;
}

.stat-item {
    text-align: center;
}

.stat-number {
    font-size: 2.5rem;
    font-weight:  700;
    display: block;
    background: linear-gradient(45deg, #FFD700, #FFA500);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    background-clip: text;
}

.stat-label {
    font-size: 1rem;
    opacity: 0.8;
}

.courses-filter-section {
    background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
    padding: 2rem;
    border-radius: 20px;
    margin-bottom: 3rem;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);
}

.filter-title {
    font-size: 1.5rem;
    font-weight: 600;
    margin-bottom: 1.5rem;
    color: #2c3e50;
}

.course-card {
    background: white;
    border-radius: 20px;
    overflow: hidden;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);
    transition: all 0.3s ease;
    margin-bottom: 2rem;
```

```
    position: relative;
}

.course-card::before {
    content: '';
    position: absolute;
    top: 0;
    left: 0;
    right: 0;
    height: 5px;
    background: linear-gradient(45deg, #667eea, #764ba2);
}

.course-card:hover {
    transform: translateY(-10px);
    box-shadow: 0 20px 40px rgba(0, 0, 0, 0.15);
}

.course-header {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
    padding: 1.5rem;
    position: relative;
}

.course-code {
    background: rgba(255, 255, 255, 0.2);
    padding: 0.3rem 0.8rem;
    border-radius: 15px;
    font-size: 0.9rem;
    font-weight: 600;
    display: inline-block;
    margin-bottom: 0.5rem;
}

.course-name {
    font-size: 1.5rem;
    font-weight: 700;
    margin-bottom: 0.5rem;
}

.course-department {
    opacity: 0.9;
    font-size: 1rem;
}

.course-body {
    padding: 1.5rem;
}

.course-info {
    display: flex;
```

```
    justify-content: space-between;
    align-items: center;
    margin-bottom: 1rem;
}

.info-item {
    display: flex;
    align-items: center;
    gap: 0.5rem;
    color: #666;
}

.info-item i {
    color: #667eea;
    font-size: 1.1rem;
}

.course-description {
    color: #666;
    line-height: 1.6;
    margin-bottom: 1.5rem;
}

.course-actions {
    display: flex;
    gap: 0.5rem;
    flex-wrap: wrap;
}

.btn-course {
    padding: 0.5rem 1rem;
    border-radius: 25px;
    font-size: 0.9rem;
    font-weight: 600;
    text-decoration: none;
    transition: all 0.3s ease;
    border: none;
    cursor: pointer;
}

.btn-view {
    background: linear-gradient(45deg, #667eea, #764ba2);
    color: white;
}

.btn-view:hover {
    transform: translateY(-2px);
    box-shadow: 0 5px 15px rgba(102, 126, 234, 0.3);
    color: white;
    text-decoration: none;
}
```

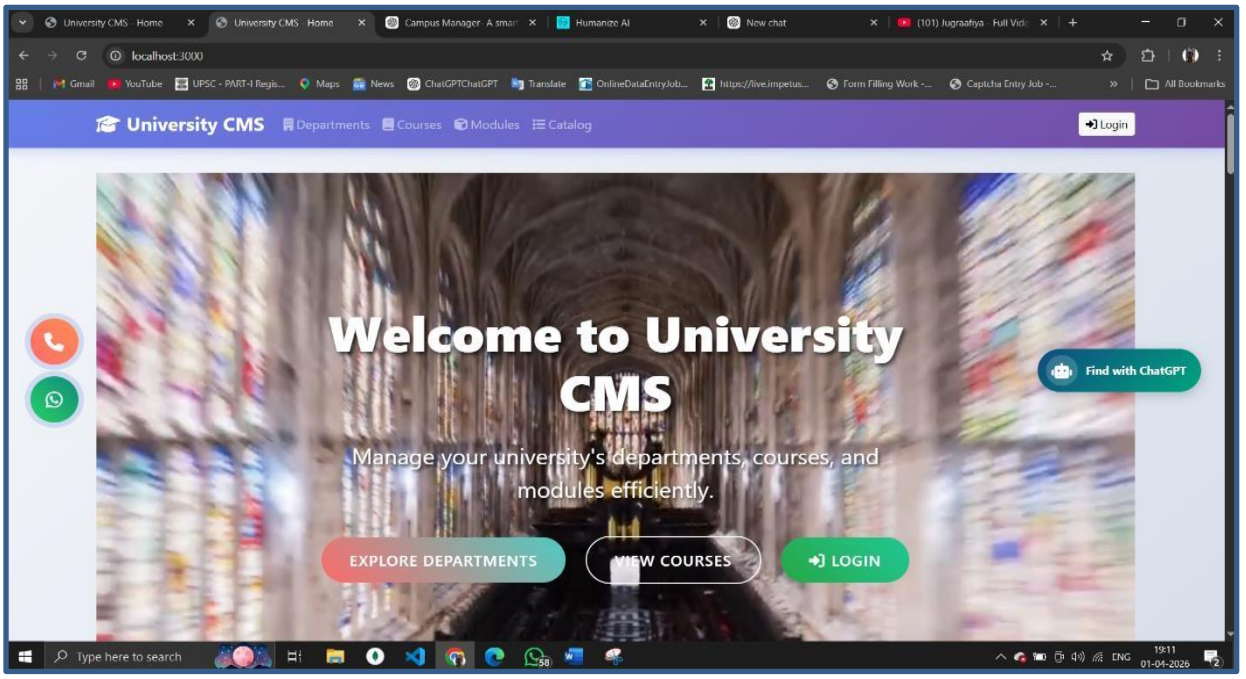
```
.btn-edit {
    background: linear-gradient(45deg, #feca57, #ff9ff3);
    color: #2c3e50;
}

.btn-edit:hover {
    transform: translateY(-2px);
    box-shadow: 0 5px 15px rgba(254, 202, 87, 0.3);
    color: #2c3e50;
    text-decoration: none;
}

.btn-delete {
    background: linear-gradient(45deg, #ff6b6b, #ee5a24);
    color: white;
}

.btn-delete:hover {
    transform: translateY(-2px);
    box-shadow: 0 5px 15px rgba(255, 107, 107, 0.3);
    color: white;
    text-decoration: none;
}

.no-courses {
    text-align: center;
    padding: 4rem 2rem;
    background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
    border-radius: 20px;
    margin: 2rem 0;
}

.no-courses-icon {
    font-size: 4rem;
    color: #667eea;
    margin-bottom: 1rem;
}

.no-courses-title {
    font-size: 2rem;
    font-weight: 700;
    color: #2c3e50;
    margin-bottom: 1rem;
}

.no-courses-text {
    color: #666;
    margin-bottom: 2rem;
    font-size: 1.1rem;
}

.btn-add-course {
```

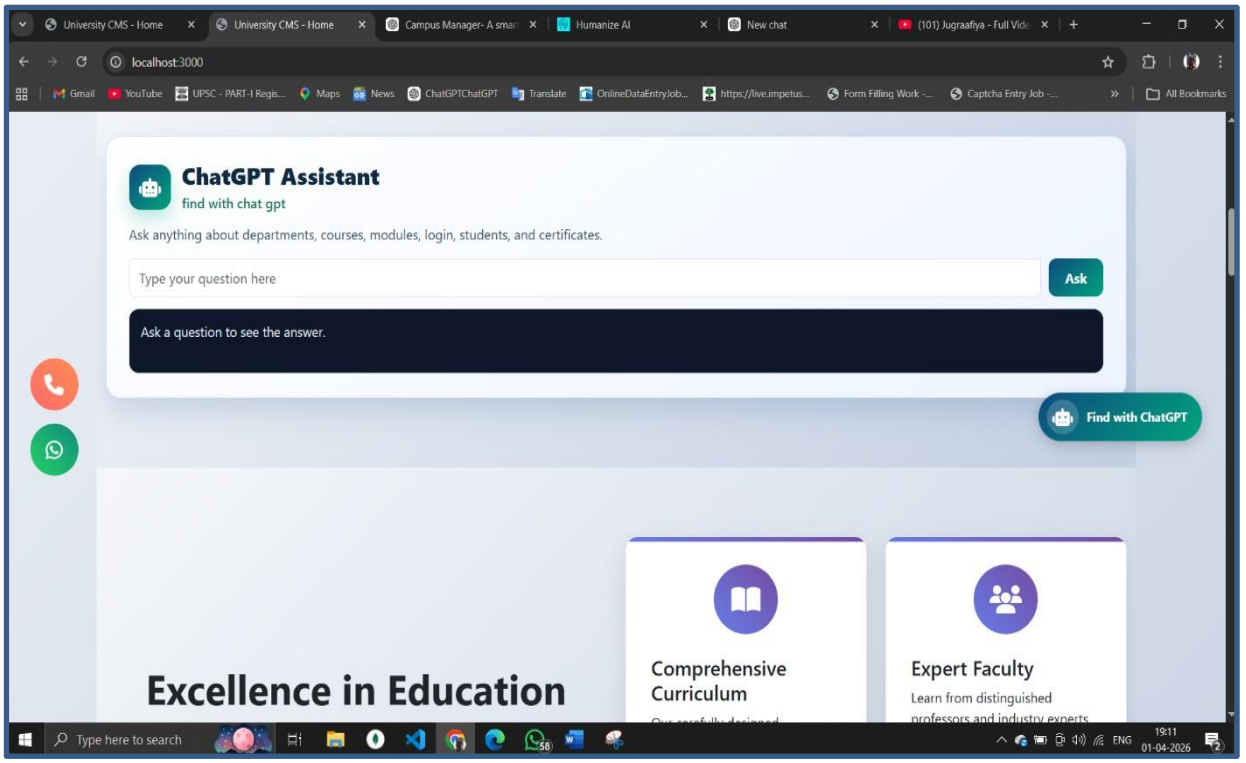
```
    background: linear-gradient(45deg, #667eea, #764ba2);
    color: white;
    padding: 1rem 2rem;
    border-radius: 30px;
    font-size: 1.1rem;
    font-weight: 600;
    text-decoration: none;
    display: inline-block;
    transition: all 0.3s ease;
}

.btn-add-course:hover {
    transform: translateY(-3px);
    box-shadow: 0 10px 25px rgba(102, 126, 234, 0.3);
    color: white;
    text-decoration: none;
}

.courses-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(350px, 1fr));
    gap: 2rem;
    margin-bottom: 3rem;
}

@media (max-width: 768px) {
    .courses-grid {
        grid-template-columns: 1fr;
    }

    .courses-stats {
        gap: 1.5rem;
    }

    .courses-title {
        font-size: 2rem;
    }
}

.jumbotron {
    background-color: #f8f9fa;
    padding: 2rem 1rem;
    margin-bottom: 2rem;
    border-radius: 0.3rem;
}

.card {
    margin-bottom: 1.5rem;
    box-shadow: 0 0.125rem 0.25rem rgba(0, 0, 0, 0.075);
    transition: transform 0.2s;
}
```

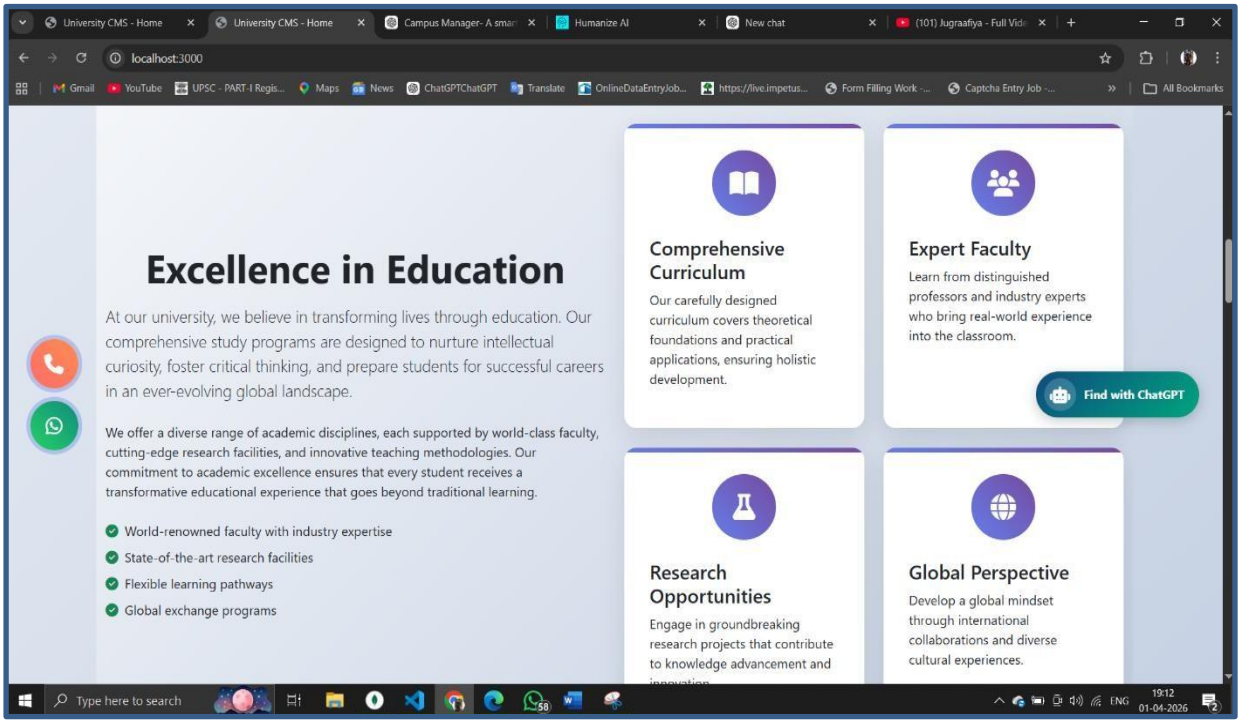
```
.card:hover {
    transform: translateY(-5px);
}

.navbar {
    margin-bottom: 2rem;
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

/* Form styles */
.form-container {
    max-width: 600px;
    margin: 0 auto;
    padding: 20px;
    background: white;
    border-radius: 8px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}

/* Table styles */
.table-container {
    background: white;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
    margin-bottom: 80px; /* To prevent footer from overlapping content */
}

/* Home ChatGPT Assistant */
.chatgpt-assistant-card {
    background: linear-gradient(145deg, #ffffff, #eef5ff);
    border: 1px solid #d9e7ff;
    border-radius: 18px;
    padding: 28px;
    box-shadow: 0 16px 34px rgba(15, 76, 129, 0.16);
}

.chatgpt-assistant-header {
    display: flex;
    align-items: center;
    gap: 14px;
    margin-bottom: 12px;
}

.chatgpt-logo-badge {
    width: 52px;
    height: 52px;
    border-radius: 14px;
    display: flex;
    align-items: center;
    justify-content: center;
    color: #ffffff;
```

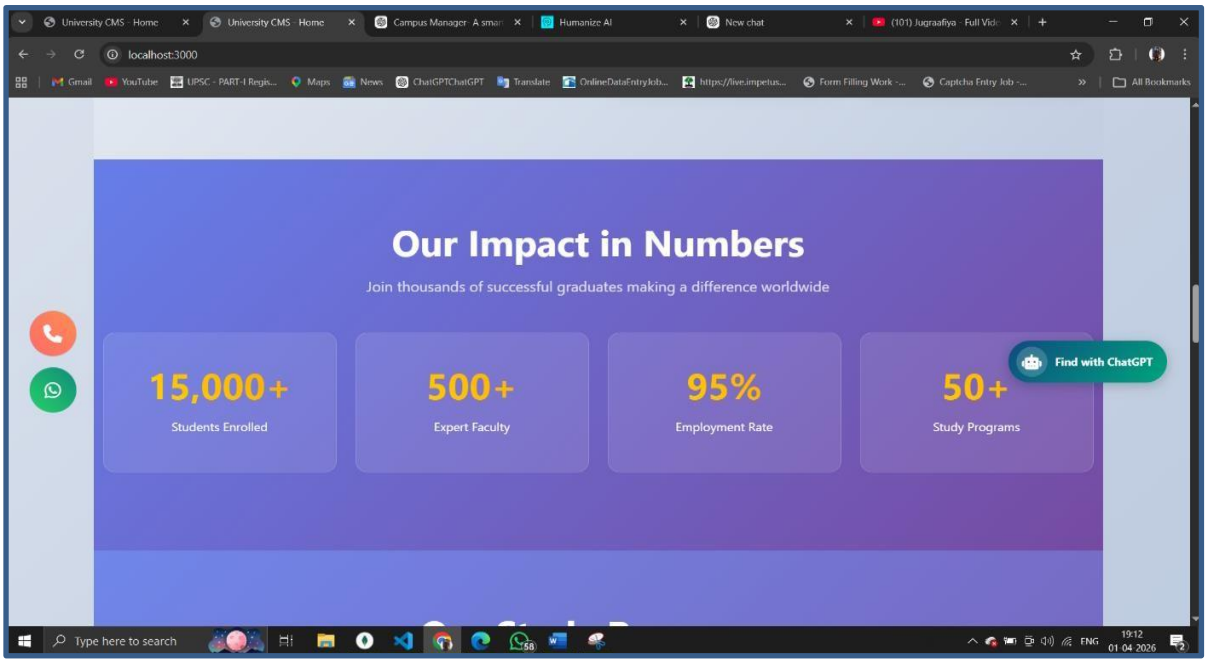
```
    font-size: 1.35rem;
    background: linear-gradient(135deg, #0f4c81, #00a67e);
    box-shadow: 0 10px 20px rgba(0, 166, 126, 0.25);
}

.chatgpt-assistant-title {
    font-size: 1.7rem;
    font-weight: 800;
    color: #102a43;
}

.chatgpt-assistant-subtitle {
    font-size: 1rem;
    color: #1f7a62;
    font-weight: 600;
    text-transform: lowercase;
}

.chatgpt-assistant-help  {
    margin-bottom: 16px;
    color: #334e68;
}

.chatgpt-input-wrap {
    display: flex;
    gap: 10px;
}
```

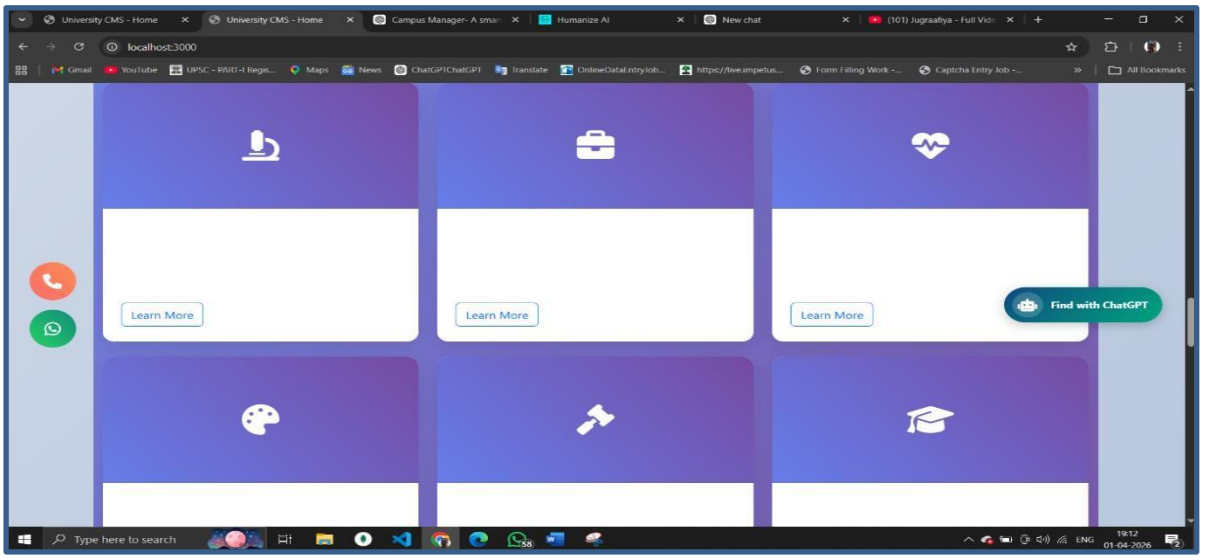
Result/Output :

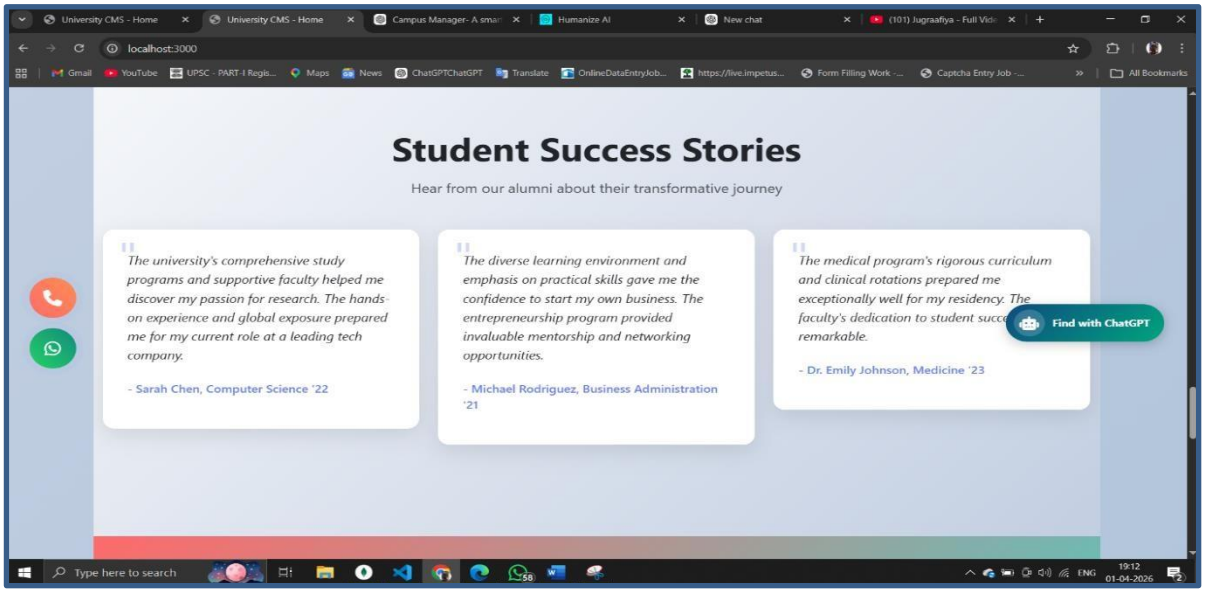


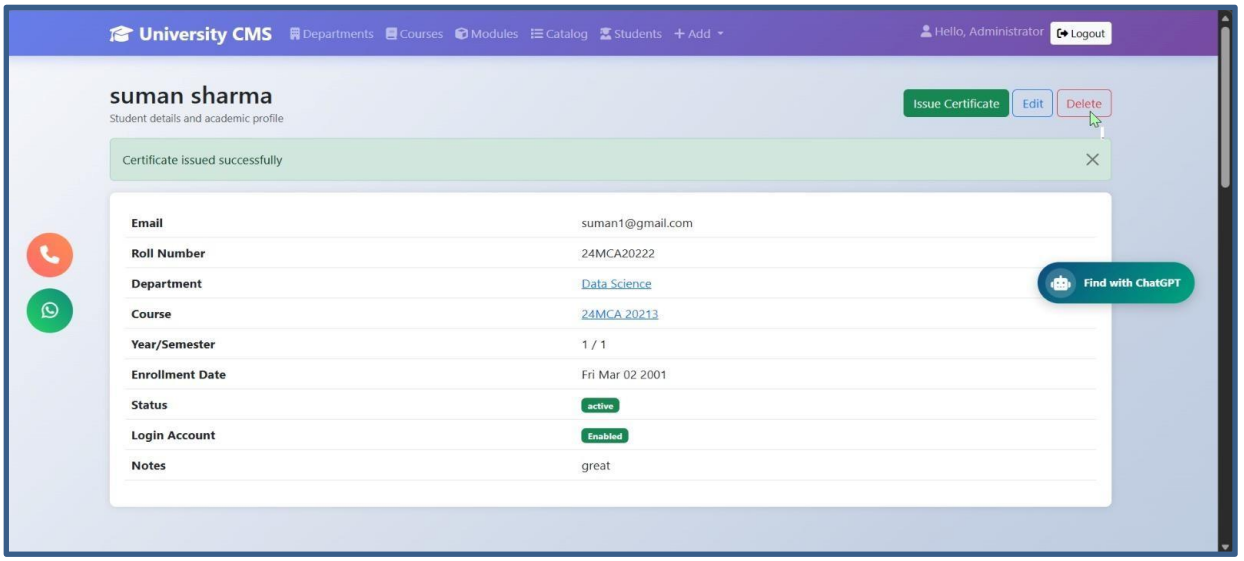

ChatGPT Assistant
find with chat gpt
Ask anything about departments, courses, modules, login, students, and certificates.
Type your question here
Ask
Ask a question to see the answer.
Find with ChatGPT
Excellence in Education
Comprehensive Curriculum
Expert Faculty
Learn from distinguished professors and industry experts


Excellence in Education
At our university, we believe in transforming lives through education. Our comprehensive study programs are designed to nurture intellectual curiosity, foster critical thinking, and prepare students for successful careers in an ever-evolving global landscape.
We offer a diverse range of academic disciplines, each supported by world-class faculty, cutting-edge research facilities, and innovative teaching methodologies. Our commitment to academic excellence ensures that every student receives a transformative educational experience that goes beyond traditional learning.
World-renowned faculty with industry expertise
State-of-the-art research facilities
Flexible learning pathways
Global exchange programs
Comprehensive Curriculum
Our carefully designed curriculum covers theoretical foundations and practical applications, ensuring holistic development.
Expert Faculty
Learn from distinguished professors and industry experts who bring real-world experience into the classroom.
Find with ChatGPT
Research Opportunities
Engage in groundbreaking research projects that contribute to knowledge advancement and
Global Perspective
Develop a global mindset through international collaborations and diverse cultural experiences.

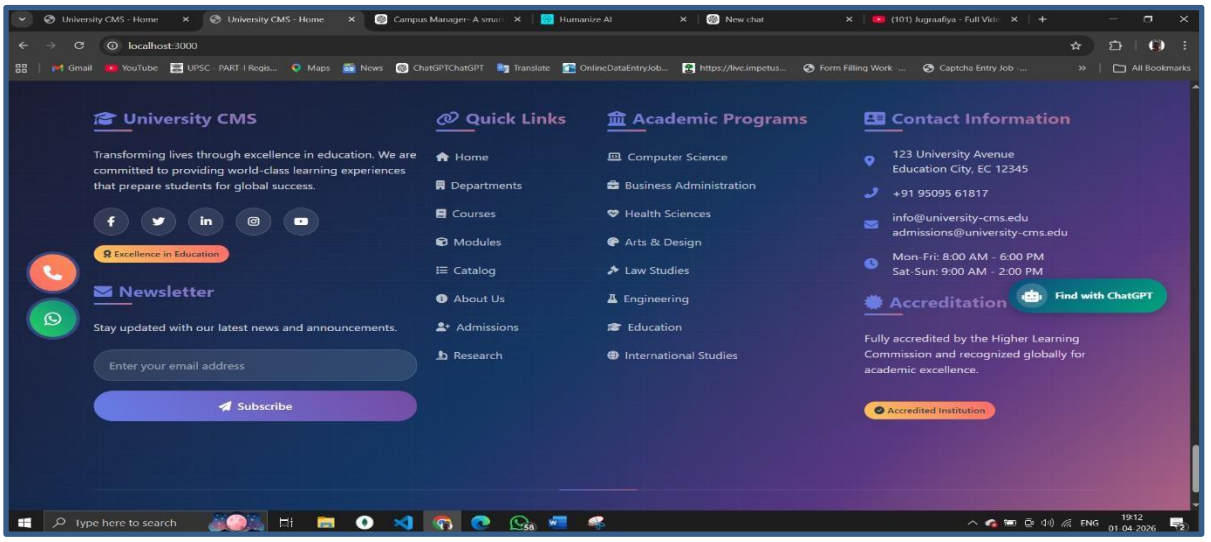

Our Impact in Numbers
Join thousands of successful graduates making a difference worldwide
15,000+
Students Enrolled
500+
Expert Faculty
95%
Employment Rate
50+
Study Programs
Find with ChatGPT


Learn More
Learn More
Learn More
Find with ChatGPT


Student Success Stories
Hear from our alumni about their transformative journey
The university's comprehensive study programs and supportive faculty helped me discover my passion for research. The hands-on experience and global exposure prepared me for my current role at a leading tech company.
- Sarah Chen, Computer Science '22
The diverse learning environment and emphasis on practical skills gave me the confidence to start my own business. The entrepreneurship program provided invaluable mentorship and networking opportunities.
- Michael Rodriguez, Business Administration '21
The medical program's rigorous curriculum and clinical rotations prepared me exceptionally well for my residency. The faculty's dedication to student succ
remarkable.
- Dr. Emily Johnson, Medicine '23
Find with ChatGPT


University CMS
Departments
Courses
Modules
Catalog
Students
+ Add
Hello, Administrator
Logout
suman sharma
Student details and academic profile
Issue Certificate
Edit
Delete
Certificate issued successfully
Email
suman1@gmail.com
Roll Number
24MCA20222
Department
Data Science
Course
24MCA 20213
Year/Semester
1 / 1
Enrollment Date
Fri Mar 02 2001
Status
active
Login Account
Enabled
Notes
great
Find with ChatGPT

UNIVERSITY COURSE MANAGEMENT SYSTEM
AWARD
CERTIFICATE
Course Completion Certificate
This is proudly presented to
suman sharma
Roll Number: 24MCA20222
for successful completion of
24MCA 20213 (22334)
Certificate Number: CERT-20260401-WU0JDR
Issue Date: 01/04/2026
Issued By: Administrator
Registrar
Authorized Signature
This certificate is system-generated and digitally verifiable.


localhost:3000
University CMS
Transforming lives through excellence in education. We are committed to providing world-class learning experiences that prepare students for global success.
Excellence in Education
Newsletter
Stay updated with our latest news and announcements.
Enter your email address
Subscribe
Quick Links
Home
Departments
Courses
Modules
Catalog
About Us
Admissions
Research
Academic Programs
Computer Science
Business Administration
Health Sciences
Arts & Design
Law Studies
Engineering
Education
International Studies
Contact Information
123 University Avenue
Education City, EC 12345
+91 95095 61817
info@university-cms.edu
admissions@university-cms.edu
Mon-Fri: 8:00 AM - 6:00 PM
Sat-Sun: 9:00 AM - 2:00 PM
Accreditation
Find with ChatGPT
Fully accredited by the Higher Learning Commission and recognized globally for academic excellence.
Accredited Institution

CHAPTER 6: RESULT ANALYSIS AND VALIDATION:

6.1 System Testing:

System testing is a vital part of the development process, and its main goal is to ensure that the Campus Manager system functions properly and meets all the specified requirements. This includes testing whether all modules are functioning as they ought to and whether they are free from errors.

Testing takes place at different levels to ensure accuracy and reliability.

Types of Testing Performed

1. Functional Testing

Functional testing helps us verify whether all functions within a system are working properly.

Login system tested with proper and improper inputs
Department, course, and module functions tested
Data storage and retrieval functions tested

2. Unit Testing

In unit testing, each module within a system is tested separately.

Authentication module tested
Course module tested
Department module tested

3. Integration Testing

In integration testing, we verify whether different modules within a system are functioning properly when combined.

Communication between Frontend and Backend tested
Communication between Backend and Database tested

4. System Testing

Complete workflow tested
End-to-end functionality verified

5. User Acceptance Testing (UAT)

The system is tested by the users to ensure that the system meets the requirements and expectations of the users.

- Easy to use interface
- Correct functionality
- Reliable

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Conclusion

The development of the Campus Manager system represents a significant step toward modernizing and digitizing campus operations. Throughout this project, the primary objective was to design and implement a centralized, efficient, and user-friendly system capable of handling various academic and administrative activities within an educational institution.

The system successfully integrates multiple functionalities such as user authentication, department management, course management, and module management into a single platform. By replacing traditional manual processes with an automated system, it reduces paperwork, minimizes human errors, and improves overall efficiency.

One of the key achievements of the project is the implementation of a centralized database, which ensures consistency, accuracy, and easy access to data. The use of modern web technologies such as HTML, CSS, JavaScript, Node.js, and MongoDB has enabled the development of a scalable and high-performance system.

The system architecture, based on a three-tier model, ensures proper separation of concerns, making the system easy to maintain and extend. The modular design approach further enhances flexibility, allowing new features to be added without affecting existing functionalities.

Another important aspect of the system is its user-friendly interface, which ensures that users with different levels of technical expertise can interact with the system easily. The role-based access control mechanism enhances security by restricting access to authorized users only.

The testing and validation processes confirm that the system performs efficiently and meets all functional and non-functional requirements. The system demonstrates fast response times, reliable performance, and accurate data handling.

Overall, the Campus Manager system achieves its goal of providing a comprehensive solution for managing campus operations. It improves productivity, enhances communication, and provides a structured approach to handling institutional data.

In conclusion, the project demonstrates the practical application of full-stack web development and highlights the importance of integrating technology into educational systems. The Campus Manager system serves as a foundation for developing more advanced and intelligent campus solutions in the future.

7.2 Future Enhancements

While the Campus Manager system successfully meets its current objectives, there are several opportunities for future enhancements that can further improve its functionality, performance, and usability.

1. Mobile Application Integration

A mobile application can be developed to provide users with easy access to the system on smartphones and tablets. This will allow students, faculty, and administrators to access information anytime and anywhere.

2. Cloud Deployment

Deploying the system on cloud platforms such as AWS or Azure can improve scalability, reliability, and accessibility. Cloud deployment also enables better data backup and disaster recovery.

3. Advanced Analytics and Reporting

Future versions of the system can include advanced analytics features to generate reports and insights based on data. This will help administrators make informed decisions.

4. AI-Based Features

Artificial Intelligence can be integrated to enhance system capabilities, such as:

- Predicting student performance
- Automating attendance tracking
- Providing personalized recommendations

5. Online Attendance System

An automated attendance system using technologies such as facial recognition or QR codes can be implemented to improve accuracy and efficiency.

6. Integration with Other Systems

The system can be integrated with other institutional systems such as:

- Library management system
- Hostel management system
- Fee management system

This will create a complete campus management solution.

7. Enhanced Security Features

Future improvements can include:

- Data encryption
- Secure API communication

8. Notification System

A real-time notification system can be added to provide updates about:

- Announcements
- Course updates

- Important events

9. Multi-Language Support

Adding support for multiple languages will make the system accessible to a wider audience.

10. Performance Optimization

Further optimization techniques can be implemented to improve system performance, especially when handling large datasets and multiple users.

Github :- https://github.com/dzcoder21/campus-manager-cms